



MDMC

Master in Data Management
and Curation

MASTER IN DATA MANAGEMENT AND CURATION

Building Service Layers in the NFFA-DI Digital Ecosystem: Governed Bucket Management and Reusable Analysis Services

Supervisors:

Dr. Federica BAZZOCCHI,
Dr. Tommaso RODANI

Candidate:

Luis Fernando PALACIOS FLORES

2025–2026





This course has been funded by the European Union – NextGenerationEU within the projects:

- *NFFA-DI cod. IR00000015 - Missione 4, “Istruzione e Ricerca” – Componente 2, “Dalla ricerca all’impresa” – Linea di investimento 3.1, “Fondo per la realizzazione di un sistema integrato di infrastrutture di ricerca e innovazione” – Azione 3.1.1, “Creazione di nuove IR o potenziamento di quelle esistenti che concorrono agli obiettivi di Eccellenza Scientifica di Horizon Europe e costituzione di reti” (CUP B53C22004310006)*

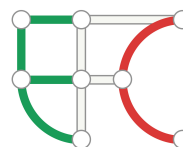
[NFFA-DI : Nano Foundries Fine Analysis - Digital Infrastructure](#)

- *EFC cod. SSU2024 – 00002 - Missione 4, “Istruzione e Ricerca” – Componente 1, “Potenziamento dell’offerta dei servizi all’istruzione: dagli asili nido all’ universita” – Linea di investimento 3.4, “Didattica e competenze universitarie avanzate” – Sub-Investimento, “Rafforzamento delle scuole universitarie superiori” (CUP G97G24000100007)*

[EFC : Educating Future Citizen](#)



nffa-di



Author's Declaration

I, Luis Fernando Palacios Flores, declare that this thesis entitled, 'Building Service Layers in the NFFA-DI Digital Ecosystem: Governed Bucket Management and Reusable Analysis Services,' and the work presented therein are my own.

I certify that:

- This work was performed principally during the MDMC internship at the Laboratory of Data Engineering, AREA Science Park.
- If any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have relied on the published work of others, this is always clearly attributed.
- Where the thesis is based on work I have done in collaboration with others, I have made clear exactly what was done by others and what I contributed.
- With respect to AI technologies, I acknowledge the use of [Codex](#), [Claude Code](#), and [Gemini](#) to support code implementation and debugging, background research, understanding of the technologies used in the thesis, and revision of the manuscript. Final decisions, verification, wording, and conclusions remain my responsibility. No AI-generated content is presented as a substitute for my own work.
- Where I have quoted from the work of others, the source is always cited. Except for such quotations, this thesis is entirely my own work.
- I have acknowledged all major sources of help.

Signed:

To every unsatisfied soul that doesn't stop its movement.

Acknowledgments

I gratefully acknowledge the study scholarship from AREA Science Park for the FAIR-by-design track of the MDMC program, as well as the participation-fee scholarship awarded by SISSA. I thank SISSA and AREA Science Park for giving me the opportunity to follow my curiosity and immerse myself in a field I genuinely wanted to understand.

I sincerely thank Dr. Federica Bazzocchi for her supervision and for the meetings that helped me better contextualize the Bucket Manager web application project. I enjoyed designing the application and thinking about how to provide a clear and useful user experience. I also thank Marco Cessarato for his guidance on the web-application design and for his support in using the RGWSquared microservice developed by him and used in this work. I hope that our interactions also contributed to making the microservice more useful for the long-term interests of AREA Science Park.

I am also grateful to Dr. Tommaso Rodani for his guidance, thoughtful suggestions, and feedback, which greatly improved the quality of the reusable analysis-service project developed through the SEM Classifier API case study.

My thanks also go to Niccolò Tosato, Isac Pasianotto, Nicola Perin and Gianfranco Gallizia for helpful conversations during the development of this thesis work.

I would also like to thank the Laboratory of Data Engineering at AREA Science Park for the supportive and welcoming environment I experienced during my time there, from the culmination of my previous master's degree to the MDMC program and the completion of this work.

Finally, my deepest gratitude goes to my family and friends for their constant encouragement and emotional support throughout this journey far from home.

Abstract

Materials science laboratories increasingly produce structured digital data, but Findable, Accessible, Interoperable, and Reusable (FAIR) data management depends on what happens after acquisition: data must enter governed storage, remain accessible under explicit policy, and become available to later computational workflows. This thesis addresses that post-acquisition problem in Nano Foundries and Fine Analysis – Digital Infrastructure (NFFA-DI) and its Overarching FAIR Ecosystem for Data (OFED), hosted on Open Research Facility for Epigenomics and Other (ORFEO), AREA Science Park’s modern computing data center in Trieste. It asks how OFED can expose governed storage and reusable analysis above FAIR-by-design laboratory pipelines. Two independent engineering projects answer that question from different points in the data life cycle. The first is the Bucket Manager, a web application that turns object storage at ORFEO into a governed workspace for proposal buckets, which are tied to formal research allocations, and local buckets, which are created directly for laboratory work. Each bucket type receives explicit ownership, access, and sharing rules. The second is the SEM Classifier API, an asynchronous Scanning Electron Microscopy (SEM) image-classification service that separates gateway, model execution, queue, and usage tracking so that model-specific logic can change without redesigning the surrounding system. Together, the two systems show that FAIR-oriented data management is not achieved by infrastructure alone: it requires deployed interfaces that encode policy, hide operational complexity, and keep storage and analysis usable for later workflows. The implementations are publicly available as [s3bucket-manager-app](#) and [sem-image-classifier-api](#).

List of Abbreviations

AGPL GNU Affero General Public License

API Application Programming Interface

CI/CD Continuous Integration and Continuous Deployment

CNR Consiglio Nazionale delle Ricerche

CUDA Compute Unified Device Architecture

DRF Django REST Framework

FAIR Findable, Accessible, Interoperable, and Reusable

FIB-SEM Focused Ion Beam Scanning Electron Microscopy

GPU Graphics Processing Unit

HDF5 Hierarchical Data Format version 5

JSON JavaScript Object Notation

JWKS JSON Web Key Set

JWT JSON Web Token

K3s lightweight Kubernetes distribution

ML machine learning

MVT Model-View-Template

NFFA-DI Nano Foundries and Fine Analysis – Digital Infrastructure

OAuth2 Open Authorization 2.0

OCI Open Container Initiative

OFED Overarching FAIR Ecosystem for Data

OIDC OpenID Connect

ORFEO Open Research Facility for Epigenomics and Other

ORM Object-Relational Mapper

RGW RADOS Gateway

S3 Simple Storage Service

SDK Software Development Kit

SEM Scanning Electron Microscopy

SQL Structured Query Language

SSO Single Sign-On

STENCIL Simulated Testing Environment for Nibble Cluster Infrastructure Lab

STM Scanning Tunneling Microscopy

TEM Transmission Electron Microscopy

TTL Time-To-Live

UI User Interface

UO Unità Operativa

URI Uniform Resource Identifier

URL Uniform Resource Locator

ViT Vision Transformer

Contents

List of Abbreviations	vi
1 Introduction	1
1.1 Why Service Layers Matter	1
1.2 From FAIR-by-design Pipelines to Operational Services	2
1.3 Engineering Contributions	3
1.4 Thesis Structure	4
2 OFED Context and Thesis Scope	5
2.1 NFFA-DI and OFED	5
2.2 Data Life Cycle and Thesis Positioning	5
2.3 ORFEO: The Operational Substrate	6
2.4 Goals and Scope	8
2.5 Relation to the FAIR Principles	8
3 Bucket Manager: Governed Storage Web Application	10
3.1 Object Storage Governance in OFED	10
3.1.1 Bucket Lifecycle Rules	12
3.1.2 Naming Convention and Data Policy	12
3.1.3 User Roles and Access Model	13
3.1.4 Architectural Overview	14
3.2 Tenant Contexts	14
3.2.1 NFFA-DI	14
3.2.2 Other Tenant Contexts	15
3.3 RGWSquared as Shared Storage-Control Service	16
3.4 Control Plane and Data Plane	17
3.5 Implementation Stack	18
3.5.1 Django Backend	18
3.5.2 React Frontend	19
3.5.3 Application Database	20

3.5.4	Federated Authentication and Deployment Environments	22
3.6	Application Interface	23
3.7	Relation to FAIR and the Data Life Cycle	24
3.8	Scope and Integration Path	25
4	SEM Classifier API: Reusable Analysis Service	27
4.1	Why OFED Needs Analysis Services	27
4.2	SEM Classifier as a Pipeline Instance	27
4.3	Architecture and Service Pattern	29
4.3.1	Reusable Operational Template	29
4.3.2	Controlled Public Access: Gateway and Authentication	31
4.3.3	Asynchronous Inference: BentoML and the Redis Queue	33
4.3.4	Usage Tracking	34
4.4	Container Image Build and Delivery	36
4.5	Using the Service	37
4.6	Operational Reporting	39
4.7	Relation to Storage and FAIR	40
4.8	Scope and Limits	41
5	Conclusions and Future Work	42
5.1	What the Bucket Manager Delivers	42
5.2	What the SEM Classifier API Delivers	43
5.3	Architectural Coherence Across Service Layers	43
5.4	Limits and Future Work	44
A	Technical Notes	45
A.1	Core Infrastructure	45
A.2	Bucket Manager: Technical Details	46
A.2.1	RGWSquared: Governance and Synchronization	46
A.2.2	Bucket Lifecycle and State Boundaries	48
A.2.3	Supplementary Interface Views	49
A.3	SEM Classifier API: Technical Details	51
A.3.1	Inference Class Hierarchy	51
A.3.2	Usage-Tracking Schema	52
A.3.3	Request and Response Shapes	52
A.4	Infrastructure Component Notes	53
A.4.1	Kubernetes Orchestration	53
A.5	Development Environment	54

Chapter 1

Introduction

Materials science laboratories produce increasingly rich datasets, but their value depends on what happens after acquisition. Data must be preserved, organized, shared, and made available to later computational workflows. At that point, laboratory data environments often fracture: files remain scattered across personal machines or laboratory servers, access rules are handled informally, and reuse depends on ad hoc intervention. Under those conditions, Findable, Accessible, Interoperable, and Reusable (FAIR) data management is structurally difficult to sustain.

This thesis addresses that problem in the context of Nano Foundries and Fine Analysis – Digital Infrastructure (NFFA-DI). The broader NFFA-DI program aims to move from isolated laboratory workflows to a distributed infrastructure that shares a digital ecosystem: Overarching FAIR Ecosystem for Data (OFED). Within that ecosystem, FAIR-by-design pipelines, governed storage, and advanced services support the life cycle of scientific data [1, 2, 3]. A FAIR-by-design pipeline is a laboratory-side workflow that captures, structures, validates, and prepares data near the instrument before those data enter later storage and reuse stages [1]. The first MDMC cohort developed such pipelines for instrument and process contexts [4, 5, 6, 7]. This thesis starts where those pipelines leave off: once data are prepared, the ecosystem still needs governed storage and reusable analysis above them.

1.1 Why Service Layers Matter

The FAIR principles are often presented as guidance, but an ecosystem depends on mechanisms rather than declarations. A service layer is a software boundary that turns shared infrastructure into repeatable user-facing capability: naming rules make data findable, authentication and sharing poli-

cies make access governable, and downstream services make reuse possible through controlled paths. By *governance* we mean the explicit rules that decide who can create, read, modify, share, or delete each data resource, together with the mechanisms that enforce those rules at the infrastructure layer rather than through informal agreement. FAIR therefore becomes credible only when infrastructure enforces the desired behavior.

Generic file-sharing services such as Google Drive, Dropbox, or Microsoft OneDrive solve a different problem. They give individuals and small teams a place to put files behind a login, but they do not encode the structures a research infrastructure needs: accepted proposals, instrument-scientist roles, institutional affiliation codes, or the data-policy requirements of programs such as NFFA-DI. They also place the data outside the infrastructure that researchers can audit, instrument, and connect to downstream analysis services. The goal here is not to copy those products; it is to give researchers a familiar workspace while keeping data inside the scientific infrastructure, where policy, naming, identity, and access are enforced at the storage layer itself.

This requirement is especially sharp in a distributed research infrastructure. Just as a hospital’s records system enforces who can read or update a patient file—without requiring every nurse or physician to understand the underlying database—OFED must enforce who can access or modify each data object without requiring every researcher to understand storage administration. If the interface is too technical, researchers must reason about storage endpoints, identity systems, and deployment details that are orthogonal to their scientific work; if it is missing, the ecosystem depends on technical staff and fragile handoffs. Simpler open-source front ends are feasible, but they tend to expose the underlying infrastructure directly and stay unfriendly to non-technical users. The work in this thesis takes the opposite stance: a user-facing interface sits on top of the existing infrastructure and absorbs the policy, naming, and access work that would otherwise leak into every workflow.

1.2 From FAIR-by-design Pipelines to Operational Services

Within NFFA-DI, OFED provides the setting for the work discussed here. It connects planning, storage, access control, sharing, and analysis inside one digital environment rather than treating them as unrelated tools [2, 1]. The gap addressed by this thesis begins after laboratory-side FAIR-by-design

pipelines have prepared data for the ecosystem. Those pipelines structure data close to the instruments, but the infrastructure must still provide governed destinations for shared objects, controlled access paths for users, and reusable analysis above the stored data. An operational service layer is a deployed interface that turns such infrastructure capability into a stable workflow: users interact with a consistent surface, while the system handles identity, policy, state, and execution behind it.

This gap makes the central engineering question precise:

How can *OFED* expose governed storage and reusable analysis as operational service layers above FAIR-by-design laboratory pipelines?

Two immediate problems follow from that question. Shared object storage must become usable without forcing researchers to work directly with low-level storage operations, permissions, and provisioning logic. Advanced analysis must become callable as a reusable service rather than remaining an isolated model, script, or deployment pattern. Those problems define the Bucket Manager and the SEM Classifier API projects of this thesis.

1.3 Engineering Contributions

This thesis makes three engineering contributions:

1. **A governed bucket-management service:** we built a user-friendly web application, the Bucket Manager, on top of the existing object-storage infrastructure. A *bucket* is a named container that holds a flat set of stored objects under one access policy; the application exposes bucket creation, ownership tracking, exploration, and controlled sharing through a familiar workspace interface while enforcing tenant scope, naming, and data-policy rules at the storage layer. Researchers therefore work inside one consistent surface, and the service absorbs the operational and policy details that would otherwise sit in every workflow.
2. **A reusable SEM analysis service:** the SEM Classifier API shows how an advanced image-analysis capability can be exposed through an asynchronous Application Programming Interface (*API*) rather than through an isolated machine learning (*ML*) workflow. The same pipeline can host another model by replacing the model-specific inference steps while keeping the same *API*, making this approach reusable for other analysis tasks.

3. **A service-layer positioning view:** the two projects occupy adjacent **OFED** layers after existing laboratory FAIR-by-design pipelines. Developed independently, they show that the same digital infrastructure can support governed storage and reusable analysis services at two consecutive points in the data life cycle. This positioning identifies a future integration path; that integration is future work, not a claim about the current systems.

The contribution is broader than the existence of two applications. The storage project turns policy-backed object storage into governed workflow, while the analysis project turns model inference into a callable **OFED** capability. Together they make the post-pipeline part of **OFED** concrete: prepared data can enter governed storage, and computational capability can be exposed above that storage without reducing the ecosystem to isolated scripts or administrator-driven handoffs.

1.4 Thesis Structure

Chapter 2 positions the work within **NFFA-DI** and **OFED**, clarifies the relevant data-life-cycle perspective, and relates it to the earlier MDMC pipeline work. Chapter 3 develops the Bucket Manager as a governed storage web application. Chapter 4 develops the SEM Classifier API as a reusable analysis service. Chapter 5 brings the two layers back together, states what the implementations establish, and identifies the remaining limits and next steps.

Chapter 2

OFED Context and Thesis Scope

2.1 NFFA-DI and OFED

NFFA-DI is an Italian digital infrastructure that combines instruments, computational resources, and data-oriented services across multiple operational units [1, 3]. It serves external researchers, who access facilities through accepted research proposals, and laboratory scientists, who operate instruments and manage local research activity inside the infrastructure. That organizational split shapes the data problem: research outputs should not remain confined to local laboratory workflows, but should enter a shared environment in which storage, access, planning, sharing, and later analysis are coordinated.

OFED provides that environment. In the official **NFFA-DI** documentation, **OFED** is described as more than a datalake: it is a digital ecosystem and the digital backbone of the research infrastructure [2, 1]. A datalake emphasizes accumulation. A digital ecosystem emphasizes the relationship between layers, policies, and services. **OFED** connects FAIR-by-design pipelines from the laboratories with common infrastructure below and value-added services above. The discussion here focuses on two of those layers: storage and access, and reusable analysis services.

2.2 Data Life Cycle and Thesis Positioning

A data-life-cycle view keeps the thesis organized by function rather than by software inventory. Research data lifecycle models treat management and curation as iterative: reuse can produce new planning, collection, or analysis rather than closing the chain [8, 9]. The lifecycle places the two imple-

mentations after earlier acquisition-to-ingestion work: the Bucket Manager addresses preservation, organization, and governed access, while the SEM Classifier API addresses analysis and reuse.

Storage comes first in the thesis because later analysis needs stable, policy-backed inputs; that adjacency in the data life cycle motivates the ordering. The two services do not yet form an implemented end-to-end workflow, and the chapter conclusion comes back to that boundary.

This ordering also clarifies how the work relates to the previous MDMC cohort. Earlier theses concentrated mostly on the acquisition-to-ingestion side of the ecosystem. Marella structured growth and synthesis process information for repository integration, Vigneri refined Scanning Tunneling Microscopy (STM) metadata workflows and NeXus preparation, Saadat implemented validated submission and standard-format generation for Focused Ion Beam Scanning Electron Microscopy (FIB-SEM) data, and Perin developed a FAIR-oriented Transmission Electron Microscopy (TEM) platform with object storage and identity integration [4, 5, 6, 7]. Those projects prepared data for the ecosystem. This thesis addresses what the ecosystem must provide after that preparation: governed destinations for data and reusable services that can act on them.

Figure 2.1 makes this positioning explicit: earlier FAIR-by-design work prepares the planning, collection, and processing side of the cycle, while this thesis concentrates on governed preservation, sharing, and reusable analysis above that storage layer.

2.3 ORFEO: The Operational Substrate

The projects were developed and deployed at Open Research Facility for Epigenomics and Other (ORFEO) [10], AREA Science Park’s computing and data center in Trieste. ORFEO is where OFED is implemented in practice: it provides the shared storage, identity, orchestration, and compute infrastructure on which the services described in this thesis run.

Simple Storage Service (S3), developed by Amazon [11], is the de-facto object-storage interface used here: data are stored as objects inside buckets and accessed through a standard HTTP-based API. OFED adopts this interface because it gives every service the same way of reading and writing storage, independent of which storage system sits behind it. On ORFEO, Ceph Object Gateway, also called Ceph RADOS Gateway (RGW), provides the S3-compatible gateway to the Ceph storage cluster [12].

At the architectural level, the ORFEO substrate combines Ceph RGW, Authentik as the identity-provider and Single Sign-On (SSO) layer [13], Ku-

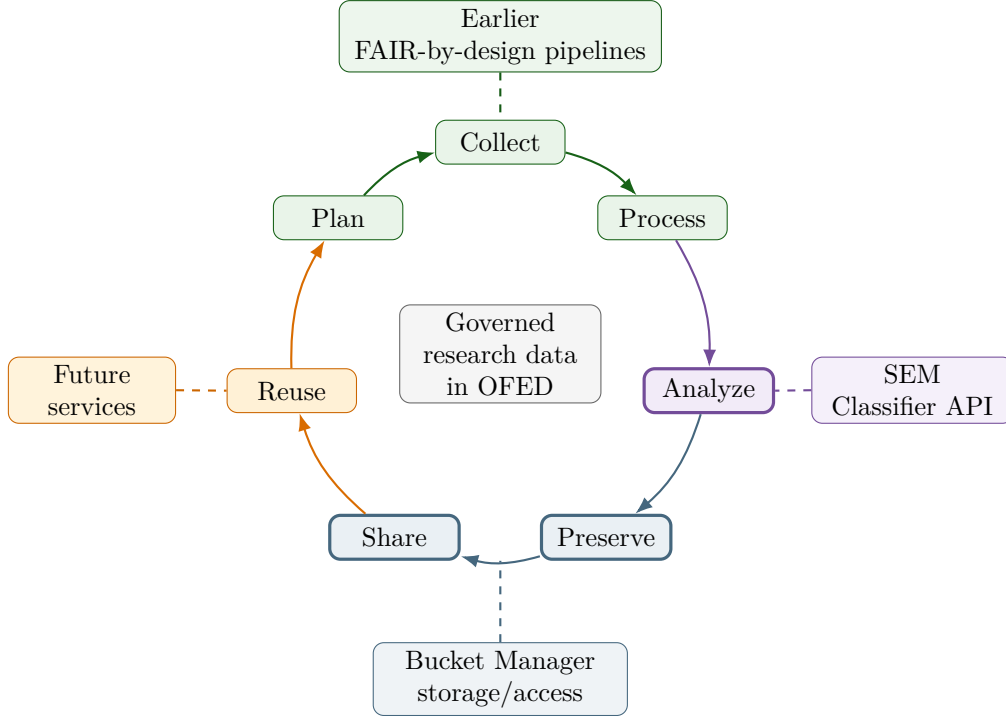


Figure 2.1: Data-life-cycle position of the two thesis projects. The cycle follows the common research-data sequence from planning through reuse and makes the feedback from reuse to new planning explicit. Earlier FAIR-by-design work prepares the planning, collection, and processing side; the Bucket Manager supports governed preservation and sharing; and the SEM Classifier API supports analysis above governed storage.

bernetes as the declarative platform that keeps multi-component containerized services in the desired state [14], and high-performance computational resources. The official **OFED** description places Ceph, Kubernetes, and Authentik in the common infrastructure layer of the ecosystem [1].

The two projects discussed here were designed against that existing environment rather than as isolated prototypes outside it. Development took place against a virtual replica of the **ORFEO** environment provisioned with Simulated Testing Environment for Nibble Cluster Infrastructure Lab (**STENCIL**), a virtual data-center framework; Appendix A.5 describes that setup in detail [15, 16].

The most useful abstraction in this environment is the *tenant*. A tenant is an organizational unit—a research program such as **NFFA-DI**, a consortium of institutes or laboratories, or an independent laboratory group—that owns a governed slice of the shared infrastructure. Within a tenant, research

activity follows one of two paths: a *proposal-driven* path, where work is part of a formal research allocation and produces a *proposal bucket*; or a *local* path, where work is organized directly by researchers inside the tenant and produces a *local bucket*. Both bucket types share the same **ORFEO** substrate but follow different governance rules, as detailed in Chapter 3.

This shared substrate also clarifies the data-life-cycle handoff introduced earlier in this chapter. Once laboratory pipelines have produced structured data, **ORFEO** is the place in which those data can be persisted under governed access rules and later consumed by higher-level services. Chapter 3 therefore addresses how prepared data enter tenant-aware storage, while Chapter 4 addresses how governed data can later be processed through an asynchronous analysis service. Reuse depends on inputs that already have stable locations, access rules, and tenant context.

2.4 Goals and Scope

The thesis has two goals. First, it defines how policy-backed object storage can become usable to researchers who should not have to reason directly about low-level **S3** operations. Access should remain user-friendly, exposed through a graphical User Interface (**UI**) that hides the storage administration beneath it. Second, it defines how analytical functionality can be exposed above governed storage, using the SEM Classifier API as a concrete implementation.

The discussion stays at the level of governance logic, user-facing interfaces, and architectural roles. Cluster administration, Ceph internals, and model-deployment detail appear only where they change the meaning of the system being described.

2.5 Relation to the FAIR Principles

The two projects contribute to different aspects of the **FAIR** principles. The storage project contributes most directly to findability and accessibility by enforcing naming logic, structured management, and authenticated access policies. This is consistent with the original **FAIR** formulation and the **NFFA-DI** Research Data Policy, which requires managed storage on **OFED**, user-specific access permissions, and explicit naming conventions for buckets and files [17, 3]. The Bucket Manager is not a full scientific repository by itself, but it makes the storage layer governable and usable inside a broader ecosystem.

The analysis project contributes from a different angle. It supports reuse and service-level accessibility by turning analysis logic into a stable, callable service rather than leaving it embedded in local workflows. That role also matches the **OFED** framing, where analysis tools appear as part of the value-added services layered above the common infrastructure [2, 1]. Storage and analysis are therefore treated as connected **OFED** layers rather than as unrelated applications.

Chapter 3

Bucket Manager: Governed Storage Web Application

3.1 Object Storage Governance in OFED

Once FAIR-by-design pipelines have captured, structured, and validated data near the instrument, the next question is where those data should go. For [OFED](#), this is not a secondary implementation detail. A digital ecosystem can only function if prepared data enter governed storage, that is, storage whose names, ownership, permissions, lifecycle, and access paths are controlled by explicit policy rather than by ad hoc file exchange. The [NFFA-DI](#) Research Data Policy makes this requirement concrete by prescribing storage on [OFED](#), user-specific permissions, and naming conventions for buckets and files [3]. Without an operational storage layer, the pipeline remains incomplete.

The deployed Bucket Manager uses [NFFA-DI](#) as its concrete tenant, while the data model can represent other tenant contexts on the same [ORFEO](#) storage substrate. Within a tenant—the organizational unit introduced in Chapter 2—storage resources divide into two types: *proposal buckets*, linked to formally accepted research projects and provisioned from upstream governance state, and *local buckets*, created interactively by researchers for work outside the formal proposal pipeline. The storage layer must manage both types without collapsing their governance paths.

Part of that governance was already in place at [ORFEO](#) for proposal-backed [NFFA-DI](#) storage. RGWSquared, a shared storage-control microservice that synchronizes tenant and proposal governance into governed storage state, together with Ceph Object Gateway (Ceph [RGW](#)), Ceph’s S3-compatible object-storage gateway [12, 11], could already materialize proposal-

governed bucket state. What remained missing was a user-facing path that researchers could actually use. Local research activity across **ORFEO** tenants still needed a coherent path for bucket creation, inspection, and controlled sharing, and even proposal-governed buckets were not useful if users had to reason directly about microservice payloads, Ceph identities, or storage-admin workflows. The engineering problem is therefore narrower and sharper than “build a storage UI”: how should governed object storage be exposed so that proposal-driven and local research work become operational inside the same ecosystem?

An earlier iteration of **OFED** planning considered MinIO, an S3-compatible object storage server distributed under the GNU Affero General Public License (**AGPL**), as the intermediate storage layer. However, MinIO’s April 2025 product restructuring moved **SSO** and OpenID Connect (**OIDC**) identity federation capabilities—previously available in the community edition—exclusively to its paid enterprise product [18]. Because federated institutional authentication is a prerequisite for a multi-tenant research environment, this change eliminated the open-source path. The Bucket Manager instead targets the Ceph Object Gateway (Ceph **RGW**) directly: it is already deployed at the **ORFEO** data center, requires no additional licenses, and exposes the same S3-compatible interface. Data-plane operations—object upload, download, listing, and presigned URL generation—are executed against this endpoint using **boto3**, the official AWS Software Development Kit (**SDK**) for Python [19].

The storage contribution is visible in the operational work the application removes. Without the Bucket Manager, each researcher needs an independently managed storage credential, each project bucket must be provisioned by a storage administrator, and sharing data across institutions requires either giving a collaborator a raw S3 key or exchanging files out-of-band. The Bucket Manager resolves that burden by acting as governed middleware between the researcher and the raw object-storage substrate. It gives researchers a workspace-like interface, while the application enforces tenant scope, policy-backed naming, and identity-linked access.

The core technical problem this addresses is a *many-to-many mapping*: multiple researchers from different institutions need to collaborate across multiple data buckets, with different permission levels per bucket and per user. Without a coordination layer, this mapping either collapses to a flat model, where everyone gets the same access, or places the entire burden of credential management on storage administrators. The Bucket Manager represents ownership, sharing permissions, and bucket types as explicit records in its own database, and every storage action is validated against the user’s tenant membership and bucket role before being delegated to Ceph.

3.1.1 Bucket Lifecycle Rules

The distinction between proposal and local buckets changes the lifecycle rules of the system: it determines who may create a bucket, where the authoritative state is stored, and whether the bucket can be removed by an end user. Proposal buckets originate from structured project state. In the **NFFA-DI** case, the policy already requires dedicated buckets for accepted proposals, with explicit naming and access rules [3]. These buckets belong to the tenant's governance layer, are synchronized from the storage-control microservice, and are not treated as user-deletable resources in the web application. Keeping this distinction explicit prevents the lifecycle rules for proposal-governed and locally managed storage from becoming entangled at the application layer.

Local buckets originate from authenticated user requests inside a tenant context. They cover the cases in which a laboratory or researcher needs governed storage for work that is not created automatically by the proposal pipeline through the RGWSquared microservice. These buckets still belong to the governed environment: they are tenant-scoped, they have explicit owners, and they remain subject to access control and naming logic. What changes is their origin and lifecycle, not their relevance to FAIR-oriented storage.

A tenant or laboratory can require the two bucket types at the same time. Proposal-driven work and locally organized work may share the same Ceph substrate, but the application keeps their lifecycle rules separate rather than treating bucket type as a display label.

Figure 3.1 summarizes this split: proposal and local buckets converge on the same Ceph object-storage substrate, but their origin and lifecycle rules remain separate.

3.1.2 Naming Convention and Data Policy

The **NFFA-DI** Research Data Policy prescribes naming discipline for all storage resources [3]. The Bucket Manager implements this requirement through three naming patterns that encode bucket origin, tenant context, and institutional affiliation directly in the bucket name:

- **Proposal buckets** (any tenant): `{tenant}/{project_id}`
Example: PRP/420
- **Local buckets in **NFFA-DI****: `{tenant}/{name-surname}-{uo_code}-{project_name}`
Example: NFFADI/alice-adams-cnr-ifn-tn-bucket02

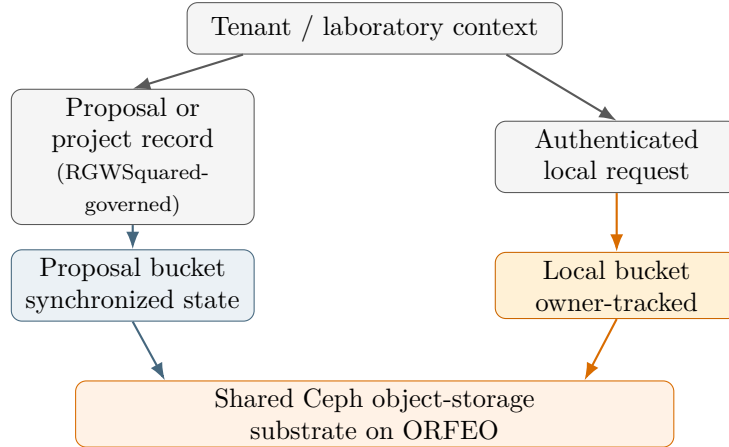


Figure 3.1: Bucket taxonomy used in the storage layer. Proposal buckets and local buckets share the same storage substrate, but they differ in origin, lifecycle, and deletion policy: proposal buckets are synchronized from governed state, while local buckets are owner-created and owner-removable in the application layer.

- **Local buckets in general tenants:** `{tenant}/{name-surname}-{project_name}`
Example: LAGE/alice-adams-localexp

For **NFFA-DI** local buckets, the `{uo_code}` segment is the institutional code listed in Table 3.1. It is absent from general local buckets, where owner identity alone is sufficient to distinguish resources, and from proposal buckets, which are identified by the proposal identifier issued upstream by the **NFFA-DI** submission system. The naming convention is enforced by the application at bucket creation time and cannot be modified by end users.

3.1.3 User Roles and Access Model

The access model distinguishes three roles within a tenant. **Researchers** receive read-only access to the proposal bucket associated with their accepted project: they can inspect and download experimental data but cannot upload files or delete objects. This role typically applies to external researchers who arrive as part of an accepted proposal—they may come from institutions different from the one operating the storage infrastructure. **Instrument scientists** are researchers affiliated with the operating institution who manage storage on behalf of the experiment; for local buckets they hold the owner role, which carries the right to delete the bucket and to add or remove collaborators. **Administrators** have cross-tenant visibility: they can

inspect all buckets, trigger a manual synchronization that refreshes user account registrations and bucket assignments from RGWSquared, and manage the mapping between identity-provider groups and tenants. The mechanics of how these rules are enforced—per-user Ceph credentials, application-database records, and the synchronization pipeline—are described in Section 3.5.

The sharing model extends beyond proposal-driven assignments. A bucket owner can share any local bucket with named users from the same tenant by assigning read-only or read-write permission per collaborator; collaborators can also leave a bucket on their own. Access is enforced in Ceph rather than only in the web interface, so sharing grants storage-level access instead of merely changing what the application displays.

3.1.4 Architectural Overview

The lifecycle rules, naming convention, and role model described above are enforced by a layered architecture. Figure 3.2 shows the resulting split: the application layer (frontend, backend, identity, application database) sits inside the ORFEO Kubernetes cluster, while RGWSquared acts as the storage-control microservice and Ceph RGW provides the S3 data plane. The remaining sections of the chapter elaborate each of these components: Section 3.2 on tenant contexts, Section 3.3 on RGWSquared, Section 3.4 on the control/data plane split, Section 3.5 on the implementation stack, and Section 3.6 on the resulting application interface.

3.2 Tenant Contexts

The storage layer serves multiple organizational contexts at once. In the ORFEO environment, research consortia, collaborative programs, and laboratory groups each need governed storage under their own namespace and access policy while sharing the same Ceph substrate. A single deployment therefore hosts NFFA-DI alongside other tenants, each operating within its own governance boundary without interference.

3.2.1 NFFA-DI

NFFA-DI is the original tenant this project targeted and the concrete reference case for the storage layer. It is a European research infrastructure built around eleven Italian operational units spanning the Consiglio Nazionale delle Ricerche (CNR) institutes, AREA Science Park, Politecnico di Milano,

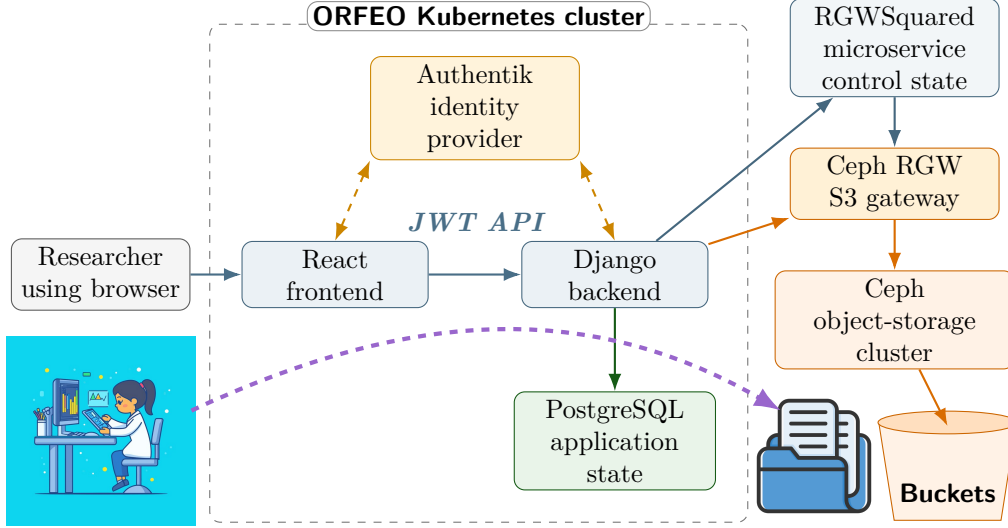


Figure 3.2: Responsibility split in the Bucket Manager architecture. The researcher uses a browser outside the **ORFEO** Kubernetes cluster; the React frontend, Django backend, Authentik, and PostgreSQL form the application layer. Django authorizes tenant and bucket operations, uses RGWSquared as the storage-control microservice, and reaches Ceph **RGW** for **S3** operations on buckets and objects stored in the Ceph cluster.

and Università degli Studi di Milano [20]. The **NFFA-DI** term for one participating unit is Unità Operativa (**UO**), for operative unit in Italian. Table 3.1 lists the code assigned to each unit. The code encodes institutional affiliation and appears directly in the naming of local buckets created within the **NFFA-DI** tenant, providing a machine-readable link between stored data and the institution that produced it [21].

3.2.2 Other Tenant Contexts

NFFA-DI is the first fully implemented tenant, but the storage architecture is not limited to it. The same **ORFEO** substrate is used by research programs and laboratory groups with their own governance logic. Each of these tenants uses the proposal/local bucket rules of Section 3.1.1: proposal-based tenants drive bucket state through the synchronization path, while local laboratory contexts rely on the owner-created bucket path. The web application's multi-tenant model accommodates these cases without changing the storage substrate: only the upstream governance source and naming convention differ. This is the basis for treating the work as a reusable storage model rather

Institution	UO code
CNR – Istituto Officina dei Materiali, Trieste	cnr-iom.ts
CNR – Istituto di Microelettronica e Microsistemi, Catania	cnr-imm.ct
CNR – Istituto per lo Studio dei Materiali Nanostrutturati, Bologna	cnr-ismn.bo
CNR – Istituto di Fotonica e Nanotecnologie, Milano	cnr-ifn.mi
CNR – Istituto di Fotonica e Nanotecnologie, Trento	cnr-ifn.tn
CNR – Istituto di Nanotecnologia, Lecce	cnr-nanotec.le
CNR – Istituto per i Superconduttori, Materiali Innovativi e Dispositivi, Napoli	cnr-spin.na
CNR – Istituto di Struttura della Materia, Roma	cnr-ism.rm
AREA – Istituto Ricerca e Innovazione Tecnologica, Trieste	area-rit
POLIMI – POLIFAB, Milano	polifab
UNIMI – Dipartimento di Fisica “Aldo Pontremoli”, Milano	unimi

Table 3.1: **NFFA-DI** operational units and their **UO** codes. The **UO** code identifies a researcher’s institutional affiliation and is embedded in the naming convention for local buckets created within the **NFFA-DI** tenant (Section 3.1.2).

than a workflow tailored to one program [22].

3.3 RGWSquared as Shared Storage-Control Service

RGWSquared is the storage-control microservice that synchronizes tenant and proposal governance state into Ceph. Before the Bucket Manager existed, RGWSquared already organized proposal-driven storage in the **NFFA-DI** environment, making proposal-governed storage reproducible rather than dependent on ad hoc administrative action. In practical terms, RGWSquared sits between upstream research-allocation logic and Ceph provisioning: it maintains tenant governance state, derives users and access relationships from synchronized state, and applies that state to the S3-compatible storage layer.

The Bucket Manager does not recreate proposal-governed storage logic inside the application. Rebuilding governance synchronization, user-level Ceph credentials, and proposal-bucket semantics inside the web application would create a second control system that could drift away from the shared

infrastructure. The application instead handles the user-facing workflow: tenant-aware actions, local ownership, sharing state, and authorization before requests reach RGWSquared or Ceph.

The source-of-truth split is therefore explicit. RGWSquared is the authoritative source for synchronized tenant governance state and proposal-governed bucket state. The application database, implemented in PostgreSQL as described in Section 3.5, is the authoritative source for local ownership, sharing state, and the user-facing workflow semantics that do not belong in the storage-control microservice. Ceph RGW is the authoritative source for the actual existence of buckets and objects. Shared synchronized governance belongs in RGWSquared, application semantics belong in PostgreSQL, and object persistence belongs in Ceph.

The project also exposed where the earlier microservice interface was incomplete. Proposal-governed synchronization already fit RGWSquared naturally, but local bucket creation and deletion initially remained direct S3 operations performed by the application with tenant management credentials. That design worked, but it placed local bucket lifecycle rules partly in the web application and partly in the storage-control layer. The integration refinement described in Appendix A.2.1 moves those lifecycle actions toward a cleaner public control-plane contract, while direct uploads, downloads, and object listing still belong on the S3 data path.

3.4 Control Plane and Data Plane

A further distinction separates who governs state from who executes storage operations. The Bucket Manager receives user intent, checks it against tenant and bucket policy, and then delegates to the lower layer that owns the relevant operation.

At the governance level, the application models tenants, memberships, bucket type, ownership, and permission state. At this level, the application enforces tenant scope and naming on each bucket, rather than treating buckets as generic S3 containers.

Two paths sit below the application. The first decides which buckets and users should exist: RGWSquared maintains the synchronized governance state on which proposal-derived users and buckets depend. The second carries the actual object traffic: Ceph RGW exposes the S3-compatible endpoint on which buckets and objects live, and uploads, downloads, listings, and presigned URL operations go directly to it via `boto3` rather than through the RGWSquared microservice, keeping high-frequency object traffic efficient [19]. For local bucket creation and deletion, the application cur-

rently uses tenant management credentials for direct [S3](#) operations; the public RGWSquared interface that will absorb these operations is described in [Appendix A.2.1](#).

Authentication is a precondition for the user-facing path, not a separate storage path. The browser first establishes identity through Authentik and receives application tokens through the backend flow. Protected storage actions then reach the backend with that token, where tenant membership, bucket ownership, and per-bucket permission checks decide whether the operation can continue.

The architecture in [Figure 3.2](#) reflects this responsibility split. The application layer connects identity, tenant membership, bucket ownership, and the requested storage action before delegating either to RGWSquared for control-plane work or to Ceph [RGW](#) for object access. Each component has a concrete responsibility. The frontend collects intent. The backend validates requests, applies tenant and ownership rules, and decides whether a bucket belongs to proposal-governed or local-governed logic. Authentik externalizes identity. PostgreSQL records application ownership and sharing state. RGWSquared holds synchronized governance information, especially for proposal-governed resources. Ceph [RGW](#) remains the storage endpoint rather than becoming part of the application itself. [Appendix A.2](#) expands this section with the model-level mapping and the separate proposal/local lifecycle flows.

3.5 Implementation Stack

The implementation stack turns the architecture in [Figure 3.2](#) into four concrete responsibilities: Django owns authorization and storage workflow, React owns the researcher-facing interface, PostgreSQL stores application state, and Authentik supplies federated identity. These choices are tied to responsibility rather than presented as interchangeable technologies.

3.5.1 Django Backend

The Django backend is the application-level authorization point for every storage operation. Django is a high-level Python web framework that follows the Model-View-Template ([MVT](#)) pattern [\[23\]](#). It provides an Object-Relational Mapper ([ORM](#)) for database access, a built-in admin interface, and Django REST Framework ([DRF](#)) as the standard extension for building typed, permission-checked JSON APIs. Django was already in use across

OFED services at ORFEO, ensuring shared operational knowledge and consistency across the ecosystem.

Beyond that institutional alignment, Django fits the specific requirements of the Bucket Manager well. The permission model—where a request must pass through tenant membership verification, bucket ownership checks, and per-bucket sharing rules before an action is permitted—maps naturally onto DRF’s layered permission architecture. The ORM maps Python class definitions to database tables and handles schema evolution through a migration system, which means changes to the domain model as the project matured were version-tracked operations rather than manual Structured Query Language (SQL) interventions. The built-in admin interface provided an immediate operational tool for managing tenants and group mappings before the dedicated administrator UI was complete.

The web application exposes four categories of functionality through the Django backend:

- **User lifecycle:** Open Authorization 2.0 (OAuth2) authentication through Authentik, automatic tenant assignment from identity-provider group membership claims, and per-tenant Ceph credential provisioning on first login.
- **Bucket management:** local bucket creation with naming validation enforced at the application layer, listing of proposal and local buckets, deletion restricted to local bucket owners, and sharing controls. Sharing lets an owner grant read-only or read-write access to members of the same tenant; collaborators receive real Ceph credentials and can also leave a bucket on their own initiative.
- **File management:** upload with per-user tracking for ownership-aware deletion, download via presigned S3 URLs, in-browser preview for images, PDF, and NeXus files (the HDF5-based open format standard for structured experimental data [24]), and deletion governed by uploader identity.
- **Administration:** cross-tenant bucket and user inspection, group-to-tenant and UO-code mapping management, and a synchronization pipeline for importing research project and researcher data from external allocation systems through RGWSquared, updating the governed Ceph state and the local application database.

3.5.2 React Frontend

React is a declarative JavaScript library for building component-based user interfaces, developed and maintained as open-source software by Meta [25]. The Bucket Manager frontend is a React application built with TypeScript for static type checking and bundled with Vite. It communicates with the Django backend exclusively through JSON Web Token (JWT)-authenticated API calls, making the entire interface a statically typed client over the same identity layer that governs backend access.

React lets the developer build a UI as a tree of independent components, each of which renders one part of the screen and re-renders automatically when its data changes. The Bucket Manager UI maps cleanly onto this structure: the dashboard, bucket detail view, file table, sharing dialog, NeXus viewer, and administrator panels are independent concerns that can be developed and updated separately. State changes—a successful file upload or a sharing permission change—propagate to the relevant parts of the UI without triggering a full page reload, which matters for a storage interface where users expect immediate feedback. The frontend also integrates H5Web [26], an open-source web viewer for HDF5 and NeXus data, which allows researchers to inspect structured experimental data directly in the browser without requiring external software.

The frontend delivers the following interface views: a *dashboard* with two bucket panels—proposal buckets synchronized from RGWSquared and local buckets owned by the researcher; a *bucket detail view* with per-file metadata, download, in-browser preview, deletion governed by uploader identity, and sharing controls for the bucket owner; a *NeXus viewer* embedding H5Web for interactive in-browser visualization of HDF5-formatted scientific data (Figure A.3); a *user profile* page displaying the user’s federated identity details and active tenant memberships; and an *administrator panel* for cross-tenant bucket and user inspection, group-to-tenant mapping management, and a step-by-step synchronization interface that coordinates the import of research project and researcher data from external allocation systems through RGWSquared (Figures A.5 and A.6).

3.5.3 Application Database

PostgreSQL is a relational database management system widely used for its reliability, standards compliance, and support for transactional workloads [27]. In the Bucket Manager, the PostgreSQL database acts as a local coordination and cache layer rather than a system of record. Three authoritative sources exist independently outside the application: Authentik holds identity

and tenant membership; Ceph **RGW** holds the actual existence of buckets and objects; RGWSquared holds proposal-governed tenant state. The database projects these into a fast, permission-checked local view that avoids round-trip latency to external services on every user request.

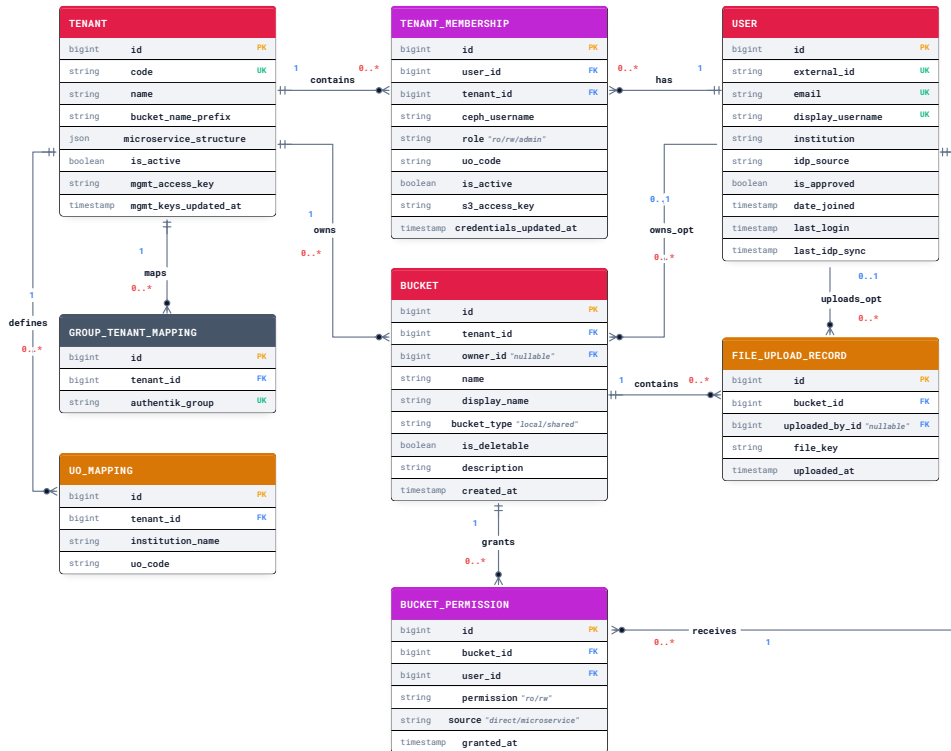


Figure 3.3: Entity-relationship diagram of the Bucket Manager application database. Seven tables organize three concerns: identity and tenancy (**USER**, **TENANT**, **TENANT_MEMBERSHIP**, **GROUP_TENANT_MAPPING**), bucket access control (**BUCKET**, **BUCKET_PERMISSION**), and supporting records (**FILE_UPLOAD_RECORD**, **UO_MAPPING**). The **TENANT_MEMBERSHIP** table caches per-user Ceph credentials with a Time-To-Live (**TTL**) to balance freshness against round-trip cost. The **UO_MAPPING** table drives the **NFFA-DI** institutional naming convention.

The schema shown in Figure 3.3 organizes seven tables around three concerns. The identity and tenancy group (**USER**, **TENANT**, **TENANT_MEMBERSHIP**, **GROUP_TENANT_MAPPING**) captures who belongs to which tenant and with which Ceph credentials. The **TENANT_MEMBERSHIP** table caches per-user Ceph access keys with a credentials timestamp, enabling Time-To-Live (**TTL**)-

based refresh that avoids regenerating credentials on every request while permitting controlled rotation. The `GROUP_TENANT_MAPPING` table links Authentik group names to tenant records, so that adding a researcher to a tenant requires only an Authentik group operation; the application reconciles membership automatically on the next login.

The bucket access-control group (`BUCKET`, `BUCKET_PERMISSION`) encodes the lifecycle and permission rules. The `BUCKET` table's `bucket_type` field (local or proposal) and `is_deletable` flag enforce the creation and deletion rules described in Section 3.1.2 without requiring a round-trip to RGWSquared at delete time. The `BUCKET_PERMISSION` table's `source` field distinguishes permissions created through the interactive sharing workflow from those synchronized from RGWSquared, which determines which permissions the application can safely remove without disturbing the control-plane state.

The supporting group (`FILE_UPLOAD_RECORD`, `UO_MAPPING`) handles per-file provenance and institutional naming. The `FILE_UPLOAD_RECORD` table tracks who uploaded each object, enabling ownership-aware file deletion in shared buckets. The `UO_MAPPING` table stores institution-to-code pairs and drives the [NFFA-DI](#) specific naming convention at local bucket creation time.

3.5.4 Federated Authentication and Deployment Environments

Access control across all tenants is built on federated identity rather than per-application credential management. The reason is simple: a researcher in a multi-institutional infrastructure should not need a separate account for every [OFED](#) service they interact with. Their institutional credentials already carry the information that matters—who they are, which institution they belong to, and which research programs they are affiliated with—and the storage layer should consume that information directly.

Authentik is an open-source identity platform that implements [OIDC](#) and [OAuth2](#) authentication flows [13]. It acts as the [SSO](#) provider that bridges institutional credentials with the Bucket Manager: researchers authenticate through their institutional identity provider, and Authentik issues signed identity tokens ([JWTs](#)) that carry group-membership claims. The Django backend reads those claims on first login and on each subsequent token refresh, synchronizing the user's tenant affiliations and per-tenant Ceph credentials without requiring any out-of-band registration step. Adding a researcher to a tenant becomes an Authentik group operation; the application state updates automatically on the next login. This removes the manual provisioning step that would otherwise fall on storage administrators and makes

onboarding scale with the size of the research infrastructure rather than with available administrative bandwidth. Appendix A.1 describes the supporting infrastructure components in more detail.

The deployment configuration differs between development and production. During development, a dedicated Authentik instance was deployed within the STENCIL-provisioned VM (see Appendix A.5) to replicate the full **OIDC** authentication flow, including group membership claims and multi-tenant routing, without depending on the production service at AREA Science Park. The development instance follows the same configuration as the production one, so the Django OAuth pipeline, the **JWT** handling, and the group-to-tenant mapping logic are exercised identically. In production, the development Authentik is replaced by the production Authentik service operated by AREA Science Park; no changes to the application code are required. Regardless of environment, a researcher who has not been registered in the Authentik service cannot authenticate: the application is insulated from unauthorized access at the identity-provider level.

3.6 Application Interface

The user-facing interface is where governed storage becomes operational: the architectural distinctions between proposal and local buckets, between owner and collaborator, and between read-only and read-write access all become visible affordances that researchers interact with directly.

Figure 3.4 shows how the dashboard makes the bucket-type distinction tangible at a glance. Proposal buckets appear under a separate heading with no delete affordance, signaling to the researcher that this storage resource is part of synchronized governance state rather than user-owned. Local buckets appear below with owner controls, including creation and deletion. Figure 3.5 shows the file-level view inside a local bucket, where collaborator visibility and per-file actions are directly exposed. Appendix A.2.3 includes supplementary screenshots: Figure A.3 shows the in-browser NeXus file viewer, Figure A.4 shows the sharing dialog, and Figures A.5 and A.6 show the administrator buckets and users panels.

3.7 Relation to FAIR and the Data Life Cycle

The storage layer supports the **FAIR** principles through mechanisms with a defined scope. It improves findability by enforcing bucket taxonomy, organization, and naming logic when buckets are created or synchronized. It

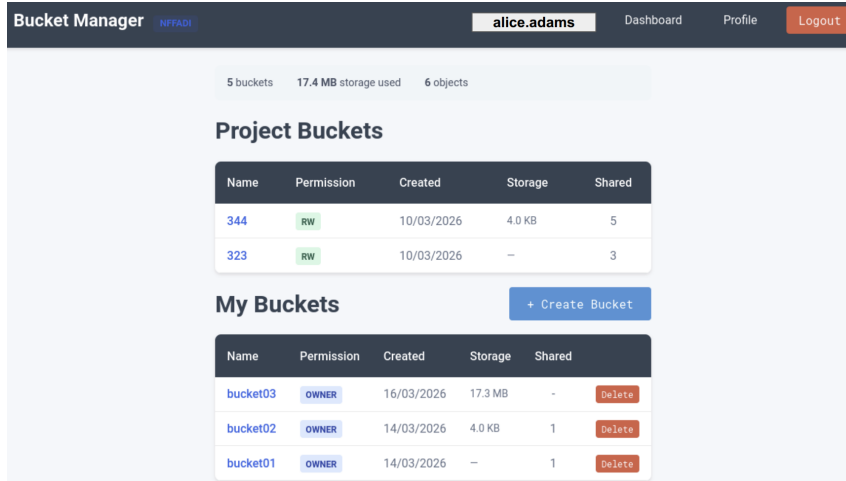


Figure 3.4: Bucket Manager dashboard for a researcher in the **NFFA-DI** tenant. The *Project Buckets* section lists proposal buckets synchronized from RGWSquared, accessible with read-write permissions but not user-deletable. The *My Buckets* section lists locally created buckets where the user holds the owner role; the **+ Create Bucket** button and per-bucket **Delete** actions are available only for this bucket type. The top bar shows the active tenant context.

improves accessibility by coupling data access to authenticated users, tenant context, and explicit sharing rules rather than informal file exchange. It supports reuse because downstream services require controlled access to governed data. Its role in interoperability is more indirect, since interoperability depends strongly on data formats and metadata standards established earlier in the FAIR-by-design pipelines [17, 3].

From the data-life-cycle perspective, this layer occupies preservation and governed access. That is why it appears before the analysis chapter. If stored data remain hard to provision, hard to inspect, or hard to share under policy, later services inherit those weaknesses. The storage layer is the destination for prepared data; by giving those data names, access rules, and tenant context, it also makes later analysis possible. For the **NFFA-DI** case, this also means turning the policy distinction between proposal buckets and locally organized work into visible system behavior rather than leaving it as documentation alone [3].

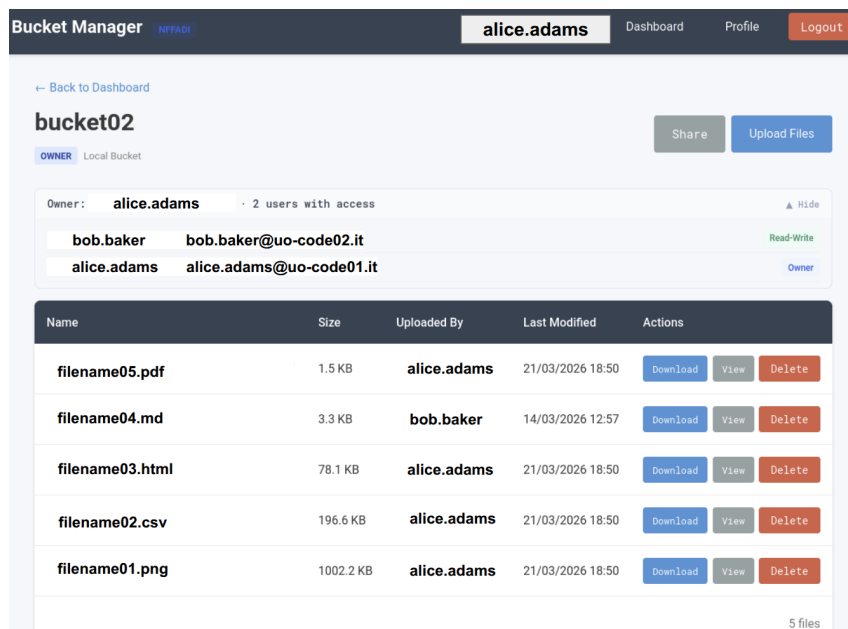


Figure 3.5: Interior view of a local bucket. The header identifies the owner and the complete list of collaborators with their assigned permission level. The file table exposes per-object metadata—size, uploader, and modification timestamp—alongside download, in-browser view, and delete actions. The *Share* and *Upload Files* controls are available to users who hold write permission on the bucket.

3.8 Scope and Integration Path

The Bucket Manager is not the final endpoint of **OFED**. It makes prepared data usable after laboratory pipelines by exposing tenant-aware bucket creation, ownership, sharing, inspection, and file access while preserving the separation between RGWSquared governance state and the Ceph **RGW** data plane. The implementation is publicly available at github.com/RitAreaSciencePark/s3bucket-manager-app.

The tenant model is designed to support multiple tenants, but only **NFFA-DI** is fully deployed in production today. The current code supports local-bucket creation and lifecycle alongside proposal-bucket workflows. Other research programs and instrument laboratories at AREA Science Park would use the same storage rules—tenant-scoped naming, ownership, permissions, and bucket lifecycle—once their governance sources are connected; the thesis therefore claims architectural generality for the service model, not complete production coverage across every **ORFEO** context.

The analysis-service project in Chapter 4 occupies the adjacent **OFED** layer. The Bucket Manager and SEM Classifier API were implemented separately: the classifier does not consume data from the Bucket Manager, and the Bucket Manager does not invoke analysis services. Their connection is architectural: governed storage gives reusable analysis services stable, policy-backed inputs. The remaining implementation boundary is also explicit: local bucket lifecycle calls currently use direct **S3** calls with tenant management credentials, while future RGWSquared public endpoints are expected to move those lifecycle operations onto a cleaner control-plane contract without changing the direct **S3** data path for uploads, downloads, and object listing.

Chapter 4

SEM Classifier API: Reusable Analysis Service

4.1 Why OFED Needs Analysis Services

Once governed storage exists, a second question becomes unavoidable: what should the ecosystem be able to do with the data it stores? **OFED** is described as a digital ecosystem precisely because it is expected to offer more than preservation and access. The official documentation places analysis tools among the value-added services that sit above the common infrastructure layer [2, 1]. A storage layer without reusable services would still leave much of the scientific workflow outside the ecosystem.

Without a shared analysis interface, every new analytical task requires its own deployment, authentication setup, and integration with stored data. In practice, this makes advanced analysis available mainly to researchers who can operate their own infrastructure, while everyone else depends on informal data transfers and one-off scripts. A shared interface gives the ecosystem a callable entry point for analytical capability.

The SEM Classifier API is the concrete instance used to develop and test the analysis-service pattern on the **ORFEO** Kubernetes cluster, which hosts current data-management and analysis services for **OFED**. The architectural pieces that make the service callable—controlled access, asynchronous execution, and usage tracking—are introduced in Section 4.3.

4.2 SEM Classifier as a Pipeline Instance

Scanning Electron Microscopy (**SEM**) is a technique that scans a focused electron beam across a sample surface to produce high-resolution images of

material microstructure. SEM is among the primary experimental modalities used across NFFA-DI facilities, and the resulting images encode information about surface morphology, composition, and material class that is not immediately legible without domain knowledge. Automating the classification of those images—assigning a material-class label to each image without human review—is therefore a recurring need in materials science workflows.

The model used here is `tOm-R/vit-sem-classification`, a Vision Transformer (ViT) fine-tuned on NFFA-DI SEM images [28]. Its class vocabulary follows the annotated nanoscience SEM dataset published by Aversa et al. [29]: biological samples, fibers, films and coated surfaces, micro-electro-mechanical-system devices and electrodes, nanowires, particles, patterned surfaces, porous sponge structures, powders, and tips. A ViT divides an input image into fixed-size patches, encodes each patch as a token, and passes the token sequence through a transformer encoder that models relationships between image regions through self-attention [30]. Fine-tuning—continuing the training of a model that was already pre-trained on a large generic dataset, but on a smaller domain-specific dataset—adapts the pre-trained image representation to discriminate between SEM image categories relevant to materials science. The result is a classifier that returns a predicted label, a confidence score, and a full probability distribution over all classes for any SEM image it receives.

This model is the *model-specific core* of the service: the preprocessing logic, the model weights, and the output schema are all SEM-specific. Swapping this model for a different fine-tuned classifier—or for a different domain entirely—would leave the rest of the service architecture (introduced in Section 4.3) unchanged. That separation is the architectural claim the chapter demonstrates.

The classifier accepts SEM images as either a Uniform Resource Locator (URL) pointing to a remote image or a binary file upload directly to the inference endpoint (the gateway address where the service receives requests, defined in Section 4.3.2). This design follows directly from the workload: images may already be stored in OFED (accessible via a URL) or provided directly by a client. A blocking request-response pattern would make the service brittle under load and difficult to share; the asynchronous pattern separates submission from execution and gives clients a stable way to interact with long-running work through job identifiers, status checks, and result retrieval.

The intended downstream path for this architecture is integration with NOMAD, an open materials-science data platform that aggregates and curates data from heterogeneous experimental and computational sources [31]. SEM image repositories stored in OFED could be passed directly to the clas-

sification service, and the resulting annotations linked back to the stored data without manual intervention. The same service shape also fits the existing SEM Classifier service exposed through the Trieste Data Portal for Nanoscience (TriDAS), the NFFA-Europe virtual-access portal [32]: re-implementing such a service with the architecture described here would keep the gateway, queue, and tracking machinery rather than rebuilding them.

The full analysis-service pipeline is publicly available at github.com/RitAr-eaSciencePark/sem-image-classifier-api.

4.3 Architecture and Service Pattern

The **ORFEO** Kubernetes cluster supplies the runtime environment for the service. Four runtime components implement the service contract, each with a distinct role. A stateless **API** gateway, KrakenD [33], controls all public access: it validates authentication tokens issued by the identity provider and routes requests to the inference backend. An inference-serving platform, BentoML [34], executes the model asynchronously. An in-memory data store, Redis [35], holds the job queue and result state between submission and retrieval by the user. A relational database, PostgreSQL [27], provides persistent usage tracking independent of job lifecycle. Kubernetes [14] orchestrates those four components by managing deployment, service discovery, and controlled communication between them.

4.3.1 Reusable Operational Template

Figure 4.1 shows the layered structure described below: the model-specific core sits inside the BentoML inference service, the gateway, queue, and tracking database form the shared service infrastructure, and Kubernetes manifests turn this composition into a repeatable deployment shape. The architecture separates the parts that change when a model changes from the parts that should remain stable for clients and operators. Three layers define that separation.

The first layer is the model-specific core. In the present project, that core is the **SEM** inference logic itself: the preprocessing assumptions for **SEM** images, the model-loading step, the inference function, and the result schema returned to the caller. This is the part that changes when the ecosystem needs a different model, a different feature extractor, or a different domain-specific post-processing stage.

The second layer is the shared service infrastructure. The public interface is held at the KrakenD gateway instead of being coupled directly to one

inference process. BentoML exposes the inference contract. Redis absorbs asynchronous workload and result-state transitions so that request submission does not depend on inference completion time. PostgreSQL records usage events separately from transient job state. Together these components define the interaction model: submit work, track job state, retrieve results later, and keep operational metadata outside the model process itself.

The third layer is the deployment shape. The gateway is produced from templated KrakenD configuration rather than handwritten one-off definitions. The multi-component topology is described through shared Kubernetes manifests—declarative configuration files that describe each component’s desired deployment state—so that gateway, inference, queue, and persistent tracking can be deployed together. The components differ in operational behavior: the gateway is lightweight and should remain continuously reachable, the inference service is heavier and may need longer startup time, and Redis plus PostgreSQL impose stability constraints that differ from the stateless public boundary. Kubernetes lets those runtime behaviors coexist inside one repeatable service pattern.

This layered structure makes the SEM Classifier API more than a single model endpoint. The design separates public access, asynchronous workload handling, operational state, and deployment lifecycle so that the same infrastructure can host a related analysis service with limited changes to the model-specific core. The part that carries over is the service form itself, not the trained model.

Figure 4.1 also explains how the service can evolve internally without breaking its public contract. The gateway absorbs request routing, JWT validation, and per-call usage recording, while the BentoML worker, Redis queue, and PostgreSQL log can each fail or restart without disturbing the client-visible interaction. The client does not need to know whether a BentoML worker has just restarted, whether a newer container image is being rolled out (a newer build replacing the current one), or whether queue pressure—temporary backlog of pending jobs—is higher than usual. Submission and result retrieval stay at the gateway layer; the readiness of the inference component is decoupled from the public entry point.

The request path starts at KrakenD; the token issuer supplies the trust anchor that lets the gateway reject unauthenticated requests before they reach BentoML.

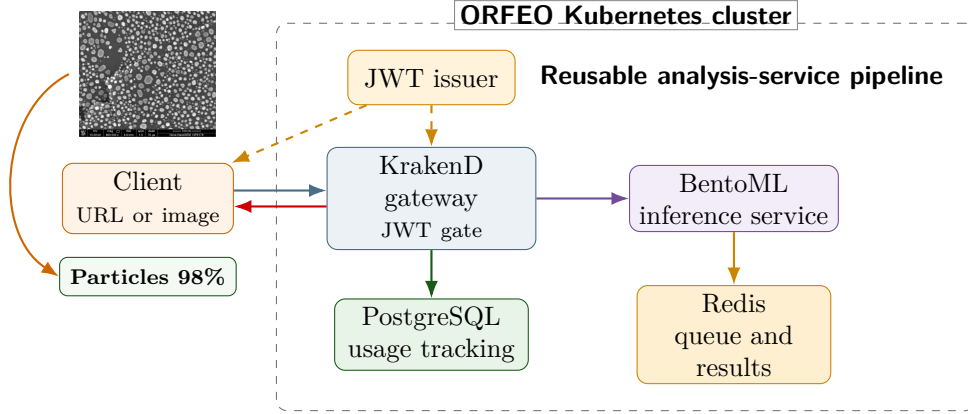


Figure 4.1: Logical architecture of the analysis-service pipeline, instantiated by the SEM Classifier API. The client submits either a URL pointing to a SEM image or an image file directly to the gateway. The client remains outside the ORFEO Kubernetes cluster; the JWT issuer handles authentication, while KrakenD validates requests before delegating work to BentoML, Redis, and PostgreSQL.

4.3.2 Controlled Public Access: Gateway and Authentication

The gateway layer has one job: to ensure that only authenticated, well-formed requests reach the inference backend. Without it, the BentoML service would be directly exposed, with no rate limiting, no token validation, and no consistent access log. KrakenD solves all three. It is stateless and fast—it adds negligible latency to the request path—and its configuration is generated from templates rather than written by hand, which keeps it consistent across environment changes.

BentoML is a Python-based inference platform whose HTTP interface treats every service call as a data submission (inputs in, outputs out), and it exposes all its service routes as POST endpoints [36]. This raw interface is functional but unauthenticated, unversioned, and without rate limiting. The gateway’s role is to wrap this interface in a coherent, versioned, and access-controlled contract. The three inference endpoints correspond directly to the three phases of the asynchronous job lifecycle: submission (**inference**), state query (**jobs/status**), and result retrieval (**jobs/results**). All three are POST, including the two that are conceptually retrieval operations. In HTTP, a GET request carries no body—parameters must be encoded in the **URL** path or query string. BentoML’s service model treats every endpoint as a body-bearing POST, so **jobs/status** and **jobs/results** receive the

`job_id` in the request body rather than in the [URL](#). This keeps the interaction pattern uniform: the client always sends a body, regardless of whether the operation submits work or queries its outcome.

Uniform Resource Identifier ([URI](#)) versioning—placing a version identifier directly in the [URL](#) path—makes the contract boundary visible without requiring clients to inspect headers or negotiate content types: a client calling `/api/v1/inference` knows exactly which contract it has bound to, and a future revision of the [API](#) can coexist at `/api/v2/` without forcing any existing client to change a call site [37]. Two additional endpoints serve infrastructure purposes: `/health` for liveness monitoring and `/api/v1/version` for [API](#) discovery. Users interact exclusively with the three versioned inference endpoints shown in Table 4.1; they never call BentoML directly.

Endpoint	Description
<code>inference</code>	Submit a SEM image (URL or binary upload); returns <code>job_id</code>
<code>jobs/status</code>	Poll job state by <code>job_id</code> ; returns status and timestamps
<code>jobs/results</code>	Retrieve classification result by <code>job_id</code> when complete

Table 4.1: User-facing inference endpoints exposed by the KrakenD gateway under the `/api/v1/` base path. All three are POST endpoints and require a valid JWT bearer token. The usage tracking plugin records every call to these three endpoints.

Authentication is enforced using [JWT](#)s signed with RS256—an asymmetric algorithm where the issuer signs with a private key and the gateway verifies using the corresponding public key, fetched from a well-known JSON Web Key Set ([JWKS](#)) endpoint where [JWT](#) issuers publish their public keys. The [JWT](#) issuer in Figure 4.1 is ghcr.io/navikt/mock-oauth2-server, a lightweight mock [OIDC](#) server used during development and testing. It issues tokens via the client-credentials flow (an OAuth 2.0 grant type in which a service authenticates with a client ID and secret rather than an interactive user login) and publishes its public key at the `/default/jwks` endpoint. Using a mock issuer allowed the full authentication path—token acquisition, bearer-token transport, [JWT](#) validation inside KrakenD—to be exercised without a production dependency on Authentik, the [OIDC](#) identity provider deployed in the [ORFEO](#) environment (see Chapter 2).

In production, replacing the mock issuer with Authentik requires changing only the [JWKS URL](#) in the KrakenD settings. The [JWT](#) validation logic, the gateway configuration, and all application code remain unchanged. The mock validates the production path without coupling development to a live identity service.

4.3.3 Asynchronous Inference: BentoML and the Redis Queue

Within BentoML’s service model, the inference logic is organized as a three-level python class hierarchy that keeps the shared infrastructure separate from the SEM-specific implementation. This separation allows the same pipeline to host a different image task, such as STM artifact detection with `t0m-R/vit-stm-artifact-fft`, or a different SEM task, such as scale classification with `t0m-R/vit-sem-scale-classifier` [38, 39]. In each case, the model-specific class changes while the submission, queueing, gateway, and usage-tracking path remain the same.

The base class `BaseAsyncModelService` owns everything that does not depend on the model domain: the Redis connection, the background worker thread, and the job-lifecycle logic. It is abstract—it defines a contract that any concrete service must satisfy, covering four responsibilities: model initialization at startup, encoding of inputs into a queue-safe representation, reconstruction of those inputs on the worker side, and execution of the model inference itself. The base class owns the queue mechanics and the background worker loop; every subclass is responsible only for what varies.

The middle class `ImageAsyncModelService` specializes this contract for image inputs. It resolves URL-based and binary-upload image submissions into a uniform in-memory representation before passing control to the base class machinery. Any image-based analysis service can inherit this layer automatically, without reimplementing the image-handling logic.

The concrete class `SEMInferenceRedisService` is the specific instantiation for SEM classification. It loads the ViT model and its image-preprocessing pipeline, runs inference, and returns the predicted label with confidence scores. Replacing it with a different image classifier requires only a new concrete class that satisfies the same four-point contract; the queue, worker loop, gateway, and usage tracking remain untouched.

The Redis queue drives the decoupled execution model. When a client submits a request, the service serializes the input—converts the in-memory data into a byte stream that can be stored or transmitted—and pushes a job record to a Redis list (the pending queue), returning a `job_id` immediately. A background worker thread running inside the same pod continuously pops jobs from that list, processes them, and writes the result back to a Redis hash—a key-value map stored under one Redis key—keyed by `job_id`. Jobs progress through states: `PENDING` → `PROCESSING` → `COMPLETED` or `FAILED`. Each job hash carries a `TTL`—a maximum lifetime after which Redis automatically expires and removes it—set to one hour by default: long enough for a client to poll and retrieve results, but bounded so that Redis memory

does not grow without limit. Periodic stale-job handling marks any job stuck in the processing state beyond a configurable threshold as failed, preventing orphaned jobs from hanging indefinitely if the worker crashes mid-inference.

In Figure 4.2, the client-visible lifecycle reduces to four operations: submit once, receive a job identifier, poll for completion, and retrieve the result when the worker has finished.

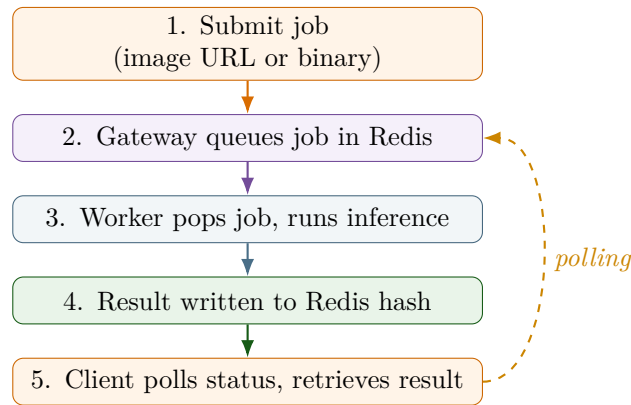


Figure 4.2: Asynchronous job lifecycle in the analysis-service pipeline. The client submits and immediately receives a job identifier; the background worker processes the job independently; the client polls for completion and retrieves the result when ready. This decoupling keeps the public interface responsive regardless of inference duration.

4.3.4 Usage Tracking

A shared analysis service that leaves no operational record is a black box from an administrative perspective: there is no way to audit who accessed it, detect unusual usage patterns, or plan capacity. The present pipeline closes this gap through a usage-tracking component implemented as a compiled plugin written in Go—Google’s statically-typed language for systems and network services [40]—embedded in the KrakenD gateway.

KrakenD exposes a plugin mechanism that lets custom logic run inside the gateway without modifying the gateway source code [41]. The plugin receives the request and response objects for configured paths and persists a usage record to PostgreSQL. It performs that database write in a Go goroutine (a lightweight concurrent task), so recording happens outside the client-facing response path [42]. Slow tracking writes can therefore affect reporting completeness, but they do not delay inference responses.

The plugin intercepts each of the three inference endpoints—`inference`, `jobs/status`, and `jobs/results` (under `/api/v1/`)—and records every authenticated call. The health and version endpoints are not tracked because they carry no inference workload and are used by infrastructure monitoring rather than researchers. For each tracked call, the plugin extracts the username from the standard `OIDC` claim path in the `JWT` payload, so any compliant identity provider supplies a usable identifier. Since the goroutine is dispatched after the response is already sent, tracking correctness remains a separate concern from service availability: a database outage silences the records but never delays the client.

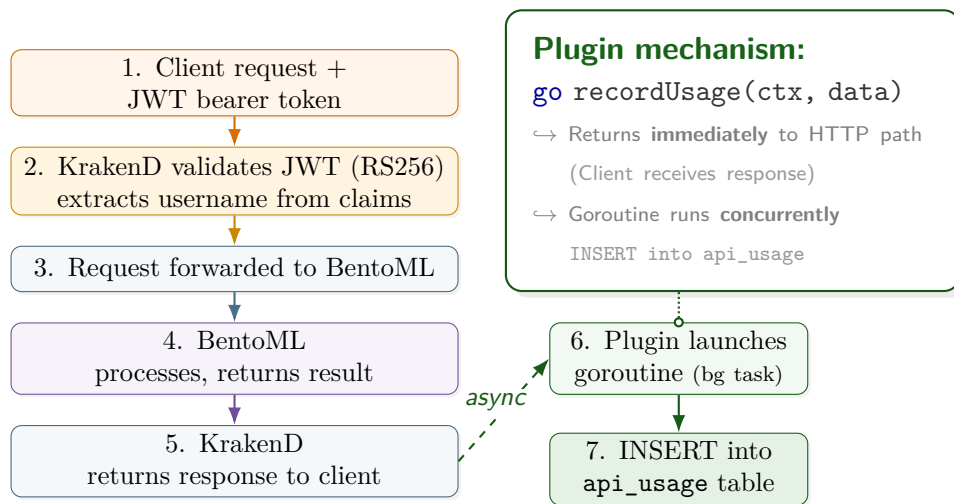


Figure 4.3: Procedural flow for a single tracked request. Steps 1–5 form the main synchronous path; the inset (upper right) details the goroutine fork at step 6. Launching `go recordUsage(...)` returns immediately to the HTTP response path, so database write latency does not appear in the client-facing response time.

Figure 4.3 shows the step-by-step sequence for a single tracked request. The schema that receives each record is a single table, `api_usage`, whose structure is shown in Figure 4.4.

Rather than storing raw URL paths, the plugin normalizes `endpoint_type` to one of three logical categories. This makes aggregate queries straightforward—counting how many inference submissions a researcher made in a given week requires a simple filter on this field rather than a pattern match on the URL string. The two indices on `timestamp` and `username` optimize the two most frequent access patterns in operational monitoring.

api_usage		
id	SERIAL	PK
timestamp	TIMESTAMPZ	NOT NULL
username	VARCHAR(250)	NOT NULL
service_name	VARCHAR(255)	
endpoint_type	VARCHAR(50)	NOT NULL
status_code	INTEGER	
url_path	TEXT	NOT NULL
Indexes: timestamp, username		

Figure 4.4: Schema of the `api_usage` table. Each row records one tracked request. The `endpoint_type` field normalizes the URL path to one of three values: `inference`, `job_status`, or `job_results`. Two indices optimize the two most frequent query patterns: time-range audits and per-user activity reports.

4.4 Container Image Build and Delivery

The inference component described in Section 4.3.3 must be packaged as a container image before it can be deployed inside the Kubernetes cluster. For stateless web applications, this is straightforward: the image contains code and configuration. For a machine-learning service, an additional question arises: where do the model weights live, and how do they get into the image? Downloading weights at pod startup (where a pod is the smallest Kubernetes deployable unit, holding one or more containers) is fragile because it adds network dependency, startup latency, and version drift; storing them outside the image requires a separate volume mount (an external storage location attached at a known filesystem path inside the container). The present project embeds weights into the image at build time so that code, dependencies, and model revision are versioned together in the container artifact.

The build uses a multi-stage Containerfile—the build-instruction file for container images, analogous to a Dockerfile but following the Open Container Initiative (OCI) specification—which installs all tooling and downloads the model weights into a local cache directory. A runtime stage copies only the resulting virtual environment and the cached weights into a clean base image, discarding all build tools. Figure 4.5 shows this flow for the two possible

model sources. Embedding the weights in the image eliminates the runtime network dependency and makes pod startup deterministic and fast. For the SEM Classifier API instance in this project, the final image is approximately 1.1 GB; the dominant contributions are the model weights, the PyTorch deep learning framework, and the base Python environment.

Model weights are fetched using one of two strategies, controlled by the `MODEL_SOURCE` build argument read from the environment configuration. When `MODEL_SOURCE` is `hugging_face`, the build downloads the model identified by `MODEL_ID` at the commit pinned by `MODEL_REVISION` directly from HuggingFace Hub (a public repository for sharing machine-learning models and their weights [43]). When `MODEL_SOURCE` is `private`, the build copies a model from a local HuggingFace cache tree into the image instead, useful when the model is proprietary or when network access to the Hub is restricted. The two strategies produce the same filesystem layout in the runtime image, keeping application code identical across public and private model-source paths and making Continuous Integration and Continuous Deployment (CI/CD) (the automated build, test, and deploy pipeline that turns committed code into a deployed service) rollouts reproducible.

PyTorch—the machine-learning framework that runs the ViT model—is installed in its CPU-only variant, which is substantially smaller than its Compute Unified Device Architecture (CUDA)-enabled counterpart (the Graphics Processing Unit (GPU)-accelerated build that requires NVIDIA GPU hardware). GPU migration requires only targeted changes to the base image, the PyTorch installation index, and the Kubernetes resource configuration—the inference code itself is hardware-agnostic. The Containerfile is written explicitly rather than generated by `bentoml containerize` (BentoML’s built-in command for producing a container image automatically), keeping the build transparent and each layer predictable.

4.5 Using the Service

A researcher or downstream service interacts with the pipeline through four steps. First, a JWT must be obtained from the identity provider: in development, this is the mock OIDC server via a client-credentials grant; in production, the equivalent Authentik endpoint. Second, the client POSTs to `/api/v1/inference` with either a JavaScript Object Notation (JSON) body containing an `image_url` field or a multipart form body containing an `image` file, along with the JWT as a bearer token in the `Authorization` header. The gateway validates the token, forwards the request to BentoML, and returns a `job_id` immediately—the response is fast regardless of how long the

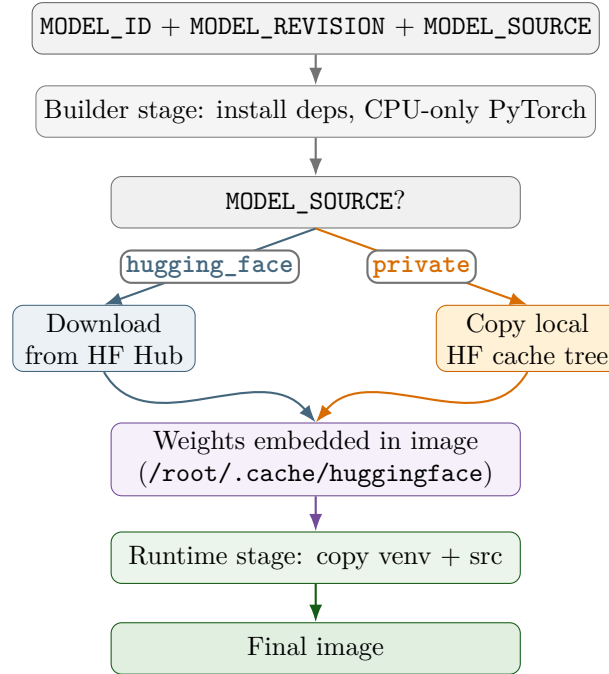


Figure 4.5: Container image build flow. Model configuration is read from environment settings; the builder stage installs dependencies and resolves the model weights via one of two paths depending on `MODEL_SOURCE`; the paths converge at an embedded HuggingFace cache; the runtime stage discards build tools and produces the final deployable image.

inference itself will take.

Third, the client polls `/api/v1/jobs/status` by POSTing the `job_id`. The response includes the current status (`PENDING`, `PROCESSING`, `COMPLETED`, or `FAILED`) and timestamps for each transition. Polling continues until the status reaches a terminal state. Fourth, the client POSTs the same `job_id` to `/api/v1/jobs/results` to retrieve the classification output: the predicted label, the confidence score, and the full probability distribution over all classes (`all_scores`).

This four-step pattern—obtain token, submit, poll, retrieve—is identical regardless of which model the service hosts. A new service built on the same pipeline exposes the same request lifecycle with only a different inference result schema. Figure 4.6 shows a concrete end-to-end run using a test SEM image.

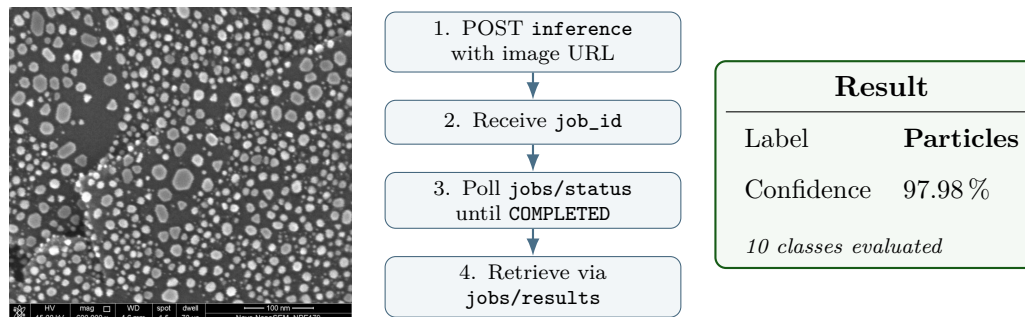


Figure 4.6: End-to-end inference example using a test SEM image of particle structures (left; image sourced from the [Nebraska Center for Materials and Nanoscience](#)). The four interaction steps (center) follow the pattern described in this section. The service returns *Particles* as the top-predicted class with 97.98 % confidence, along with scores for all ten material categories (right).

4.6 Operational Reporting

Usage data in a database only delivers value if it can be queried and interpreted. The Go plugin continuously populates the `api_usage` table (Section 4.3.4); a complementary reporting script queries it and generates structured summaries over any operator-specified time window.

The script produces three output formats. A *summary* prints a compact terminal table grouping request counts by endpoint type, status code, and username. A *JSON dump*—a text export in the structured **JSON** format—makes the raw aggregated data available for integration with external dashboards or auditing tools. An *HTML report* generates a static page with a timeline chart of requests per time bucket, a per-endpoint bar chart, a day-of-week by hour heatmap of request frequency, and a table of the most recent 25 requests. The time bucket granularity adapts automatically to the query window: minute-level for windows under four hours, hour-level for windows under a week, and day-level otherwise.

Reporting makes analysis usage inspectable after requests have been served. Governed data enter **OFED** through storage, flow through the analysis service, and generate structured usage records that can be inspected, audited, and shared with facility operators. The reporting layer is not a core architectural component—it produces no side effects on the service itself—but it makes governance visible beyond the raw database. Figure 4.7 shows a sample report generated over a development session.

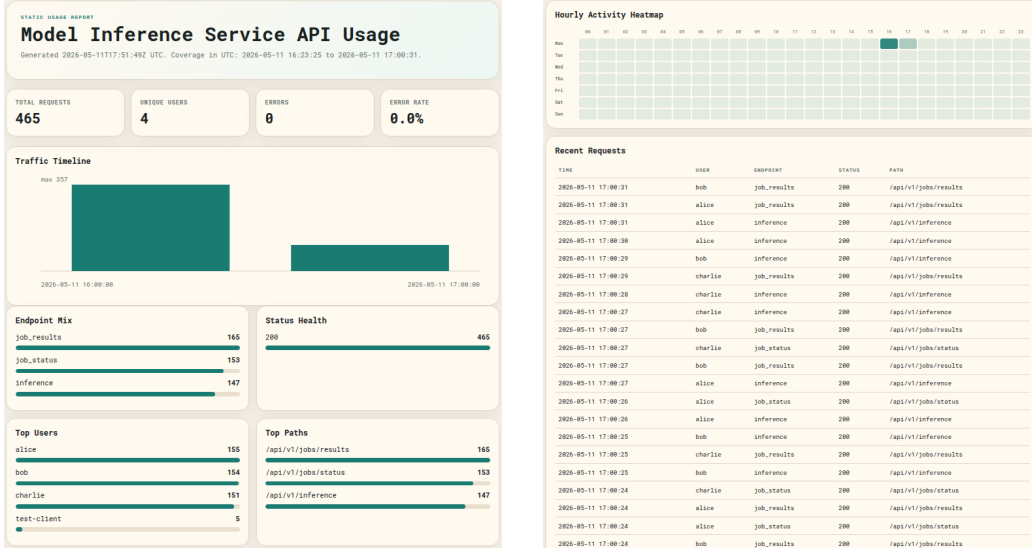


Figure 4.7: Sample HTML usage report generated from the `api_usage` table. Left: timeline of request volume per time bucket and a per-endpoint bar chart showing the distribution of calls across the three inference endpoints. Right: day-of-week by hour heatmap of request frequency and a table of the most recent requests with username and HTTP status code.

4.7 Relation to Storage and FAIR

Governed data become more useful when authorized services can consume, enrich, or interpret them through controlled access paths. The SEM Classifier API points in that direction by accepting image references or uploaded files as inputs and returning machine-generated semantic labels, confidence scores, and class distributions. Those outputs can enrich metadata around stored SEM images, making later discovery, filtering, and reuse more practical than file storage alone.

The pipeline supports reuse and service-level accessibility by turning analysis logic into a governed service. That role matches the OFED framing, where analysis tools appear as value-added services above the common infrastructure [2, 1], and it is consistent with the NFFA-DI policy emphasis on publishing analyzed data and reusable processing metadata [3]. Storage and analysis remain independent implementations in this thesis, but the service contract shows how future NFFA-DI analysis tools could add automatic semantic information to governed data without forcing each model to define a new deployment pattern.

4.8 Scope and Limits

The present implementation establishes the analysis-service pattern, not a full storage-to-analysis feedback loop. The SEM Classifier API is already a working service and implementation pattern; a later [OFED](#) stage would connect governed stored objects, analysis services, and downstream publication or visualization more tightly. That future integration remains outside the present scope, but the path toward it is clearer because storage and analysis now have defined service contracts.

Chapter 5

Conclusions and Future Work

Once FAIR-by-design laboratory pipelines produce structured data, shared infrastructure alone is not enough. **OFED** needs mechanisms that keep those data governable and computationally useful. The engineering question introduced in Chapter 1 asked how **OFED** can expose governed storage and reusable analysis above those pipelines. The two goals stated in Chapter 2—making governed object storage usable without requiring researchers to manage **S3** directly, and exposing analytical capability through a reusable service—were each addressed through a concrete implementation.

5.1 What the Bucket Manager Delivers

The storage contribution is a governed object-storage service rather than a raw storage interface. This required more than exposing Ceph through a web interface. The central architectural step was to distinguish proposal buckets from local buckets, because the two follow different governance paths even when they share the same storage substrate. Proposal buckets originate from synchronized governance state. Local buckets originate from authenticated user requests inside a tenant context. Once that distinction is explicit, the storage layer can apply different creation, deletion, and ownership rules without fragmenting the user experience.

The boundary between the Bucket Manager and RGWSquared was also made explicit through this work. RGWSquared remains responsible for synchronized governance state and proposal-governed bucket state. The Django application remains responsible for user-facing workflow, local ownership, and sharing semantics. Ceph **RGW** remains responsible for the actual persistence of buckets and objects. That responsibility split turns policy-backed storage requirements into a usable workflow while keeping the underlying control and

data planes separated. In the [NFFA-DI](#) case, this delivers concrete storage support for proposal-governed data and for local laboratory work on the same [ORFEO](#) substrate.

5.2 What the SEM Classifier API Delivers

The analysis contribution is an operational pattern for computational processing. The SEM Classifier API is the concrete instance, while the architectural result is the pipeline around it. The service separates public access, inference execution, asynchronous workload handling, and usage tracking into components that can evolve independently while remaining exposed as one coherent interface. The result is a service that can absorb workloads with variable runtime, present stable submission and result-retrieval semantics to clients, and remain understandable as an ecosystem component that can be reused.

The pipeline is relevant beyond one materials-science use case because the model-specific core can change while the surrounding components remain useful: a gateway for controlled exposure, an inference process, a queue for asynchronous execution, and persistent operational tracking. Kubernetes matters in this argument only where it preserves that deployed form. It allows the gateway, inference layer, queue, and supporting data services to be composed, updated, and exposed coherently despite their different runtime behavior. The result is an implementation pattern for how [OFED](#) can host analytical services above governed data.

5.3 Architectural Coherence Across Service Layers

Storage and analysis sit at adjacent points in the data life cycle introduced in Chapter 2, so the engineering decisions in one chapter constrain the other. The storage layer fixes where governed data live and under which access semantics they can be reached. The analysis layer builds above that floor by exposing computational capability through a controlled [API](#). The two services were implemented separately, but they follow the same operational grammar: tenant-aware identity, policy-backed access, and a user-facing contract above a more volatile internal implementation. That common shape makes them legible as two stages of one ecosystem rather than two unrelated applications.

That coherence is also what makes the result matter for MDMC. FAIR-oriented data management becomes concrete only when deployed systems enforce naming logic, access rules, lifecycle semantics, and reusable interfaces. Storage conventions, authenticated access, queue-backed analytical execution, and service composition are the mechanisms through which an ecosystem moves from policy and infrastructure into usable scientific capability.

5.4 Limits and Future Work

The limits of the thesis are implementation boundaries, not gaps in the architectural claim. Not every **ORFEO** tenant already uses the full generalized storage model in production. Local bucket lifecycle semantics are not yet fully consolidated behind the final RGWSquared public interface. The analysis-service pipeline does not yet form an end-to-end production workflow in which governed stored objects automatically feed analytical services and write enriched outputs back into the same ecosystem. The current result is therefore two explicit, deployable services rather than a finished **OFED**-wide operational pipeline.

The next step could be a tighter integration between the two layers. Governed objects stored through the Bucket Manager should become straightforward inputs for reusable analysis services, and the outputs of those services should be linked back to the same managed environment with clearer metadata and provenance. That would turn the current lifecycle sequence into a more explicit operational pipeline.

Two additional directions follow directly from the present work. The first is stronger ecosystem linkage toward visualization and publication services, so that processed results are easier to inspect, communicate, and publish without leaving the governed environment. The second is further hardening of interoperability, especially around storage-control interfaces, authentication paths, and multi-service contracts. These are extensions of the thesis result: **OFED** becomes usable when shared infrastructure is raised into well-defined storage and analysis services.

Appendix A

Technical Notes

The technical support for the main argument falls into five groups: shared infrastructure components, bucket-manager state and lifecycle mechanics, SEM Classifier API implementation details, Kubernetes orchestration, and the virtual data-center development environment. These details explain why the two projects fit the [OFED](#) ecosystem; procedures specific to [ORFEO](#) site operations remain outside scope.

A.1 Core Infrastructure

The two projects rely on the same infrastructural logic: shared identity, shared orchestration, and shared access to storage or data-adjacent services. The [OFED](#) architecture describes this common layer as the part of the ecosystem that gives coherence to otherwise different services [2, 1]. Three components shape the architecture discussed below.

Ceph Object Gateway (Ceph [RGW](#)) provides scalable, S3-compatible object storage [12, 11]. Applications interact with it using standard object-storage semantics—bucket creation, object upload and download, access control lists—without any Ceph-specific client library.

Kubernetes serves as the shared orchestration layer for the two multi-component services [14]. It gives the Bucket Manager and the analysis service stable deployment, discovery, restart, and update semantics. Section [A.4](#) covers the desired-state control model and the specific Kubernetes mechanisms used in the thesis.

Authentik provides the identity-provider and [SSO](#) layer on which governed access depends [13]. It acts as an [OIDC](#) issuer that signs identity tokens ([JWTs](#)); downstream services validate those tokens and read tenant or group-membership claims from them rather than managing user creden-

tials independently.

The main chapters refer to these technologies only where they change the meaning of the service architecture. Low-level deployment mechanics, site networking, and operator workflows stay outside the manuscript because they do not strengthen the architectural argument.

A.2 Bucket Manager: Technical Details

At implementation level, the Bucket Manager is organized around four *domain objects*—the persistent records that represent the system’s core concepts and map directly to database tables through the Django [ORM](#). A **Tenant** maps a user-facing storage domain to an RGWSquared governance record and to a tenant management identity in Ceph. A **TenantMembership** binds a user to a tenant-specific Ceph username and role. A **Bucket** stores whether a resource is proposal-based or local. A separate **BucketPermission** record stores the access mode and the origin of the permission, namely whether it was synchronized from RGWSquared or created locally through the sharing workflow.

These four objects encode the two governance paths introduced in Section [3.1.1](#): proposal-driven buckets have no ordinary end-user owner inside the application, while local buckets carry an explicit owner who can also delete them. The full database schema appears in Section [3.5](#) of Chapter [3](#); the subsections here focus on how RGWSquared feeds proposal-governed state into Ceph and how the two lifecycle paths behave in practice.

A.2.1 RGWSquared: Governance and Synchronization

RGWSquared maintains a CouchDB-backed governance model that sits between external research-allocation inputs and Ceph provisioning. In the [NFFA-DI](#) case, the relevant sources are researcher information derived from accepted proposals and laboratory-context records from instrument scientists, which together define which users and buckets should exist inside the tenant. These are combined into a governance document that is then synchronized to Ceph as users, permissions, and proposal-governed buckets. The web application consumes this synchronized view rather than attempting to regenerate it independently.

Internally, governance updates and the application of that state to Ceph are separate operations, which is why the Bucket Manager reads governance and user information from RGWSquared rather than regenerating it. The application adds the semantics that RGWSquared does not own—local own-

ership and UI-level sharing workflows—on top of the synchronized view. The newer public integration path follows the same logic: governance synchronization remains the microservice’s job, while local bucket lifecycle actions move toward a cleaner public interface rather than being handled implicitly via direct S3 calls inside the application.

The application’s interactions with the storage-control service therefore split cleanly into read-side and write-side calls. Read-side calls for tenant state discovery, user enumeration, and user details let the web application reconstruct the tenant-visible view without reimplementing Ceph internals. Write-side calls such as governance updates and local bucket lifecycle actions change the control-plane state (the synchronized governance state that decides which buckets and users should exist) and remain explicit, infrequent operations. The split keeps synchronized governance visible while avoiding the mistake of turning ordinary object traffic into storage-control service calls.

Figure A.1 shows this control path from external research-allocation inputs to RGWSquared state, Ceph governed state, and the bucket-manager view exposed after refresh.

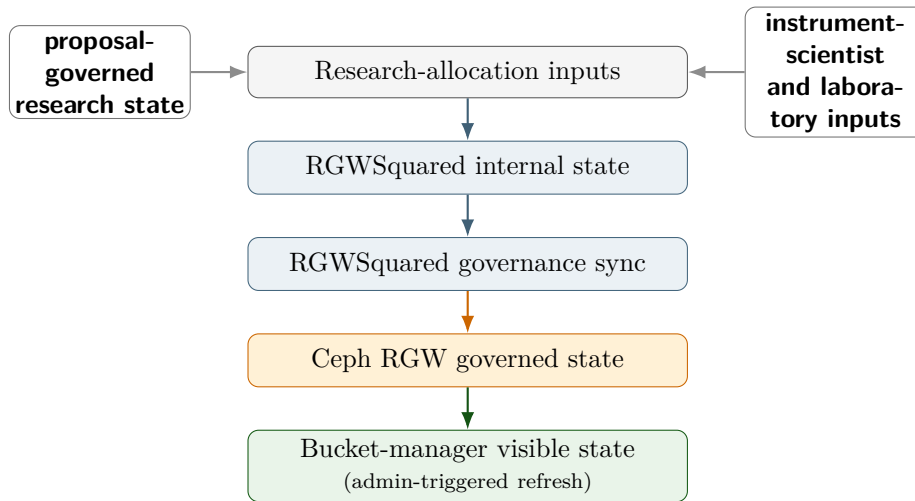


Figure A.1: RGWSquared synchronization pipeline. External research-allocation inputs are merged into RGWSquared’s internal governance state, synchronized to Ceph as governed storage state, and later exposed to users through the bucket-manager application layer after an administrator-triggered cache refresh.

A.2.2 Bucket Lifecycle and State Boundaries

Having established how synchronized governance reaches Ceph, the next question is how local and proposal buckets behave under that synchronized state. The proposal path is synchronization-oriented. An administrator-triggered governance update in RGWSquared refreshes proposal-governed users and buckets, synchronizes them to Ceph, and the web application later refreshes its local cache to expose them in the UI. The local path is interactive. An authenticated user selects a tenant, the application validates membership and naming, creates the bucket in the tenant scope, and records owner and sharing state locally. The two paths meet in the same storage substrate but they do not share the same control logic.

Figure A.2 makes the lifecycle boundary explicit by placing synchronized proposal buckets and interactive local buckets beside the shared application metadata and Ceph storage layers.

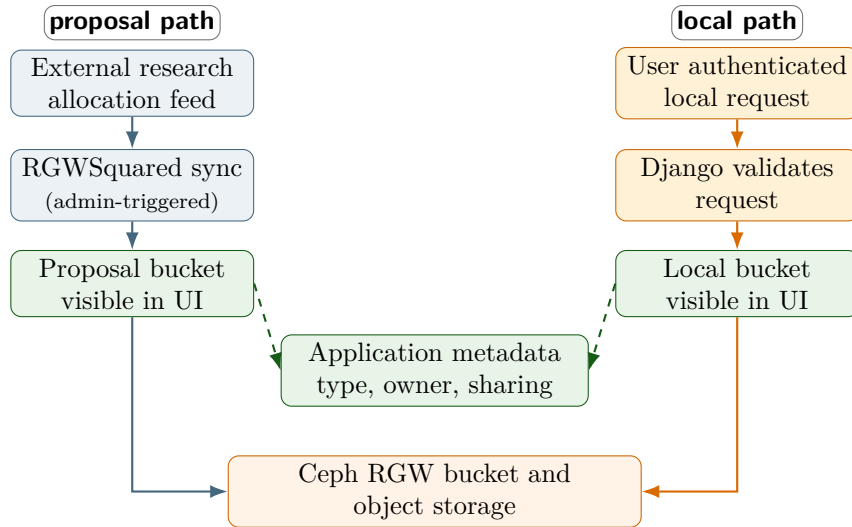


Figure A.2: Implementation-level lifecycle split in the Bucket Manager. Proposal buckets arrive through an administrator-triggered synchronization from RGWSquared, while local buckets originate from interactive user requests and are tracked with explicit ownership in the application.

This split also explains the current RGWSquared integration boundary. High-frequency file operations remain direct S3 calls through tenant management credentials because uploads, downloads, and object listing should not cross an extra microservice hop. By contrast, synchronized governance state and bucket lifecycle semantics belong naturally in RGWSquared. The ongoing public-endpoint refinement is therefore best understood as a control-plane

cleanup: governance updates remain the mechanism for proposal-governed state, while local bucket creation, deletion, and permission updates move toward cleaner microservice calls without changing the direct S3 data path.

For the **NFFA-DI** case, the policy requires different naming logic for proposal buckets and for locally organized work [3]. The application implements that distinction through bucket type, tenant-scoped naming, deletion rules, and permission origin, rather than leaving it as documentation alone.

A.2.3 Supplementary Interface Views

The remaining **UI** views complete the picture of how governance and ownership become user-visible: the in-browser NeXus viewer, the sharing dialog, and the administrator buckets and users panels.

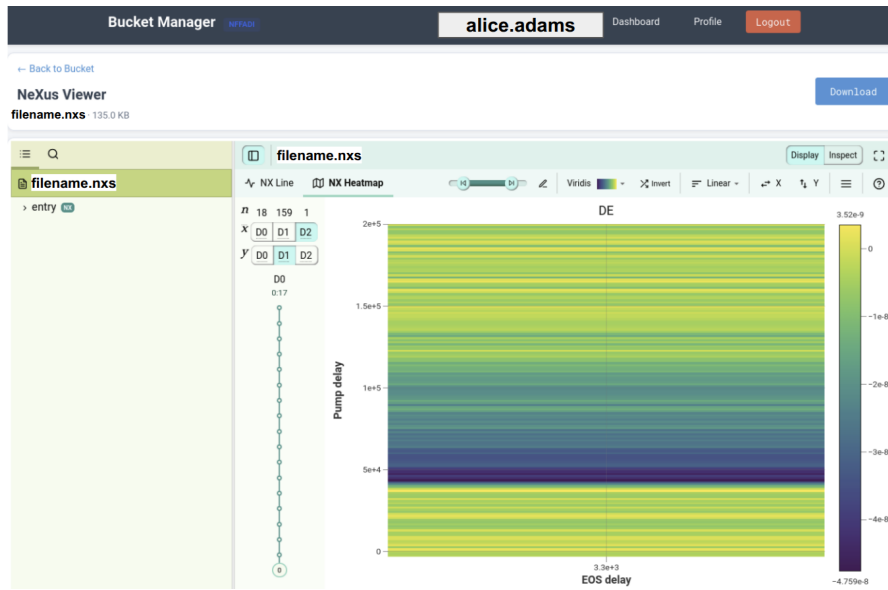


Figure A.3: In-browser NeXus file viewer. The application renders HDF5-backed NeXus files directly in the browser, displaying the file hierarchy on the left and an interactive heatmap of the selected dataset on the right. This view closes the gap between governed storage and data inspection without requiring the researcher to download and open the file with external software.

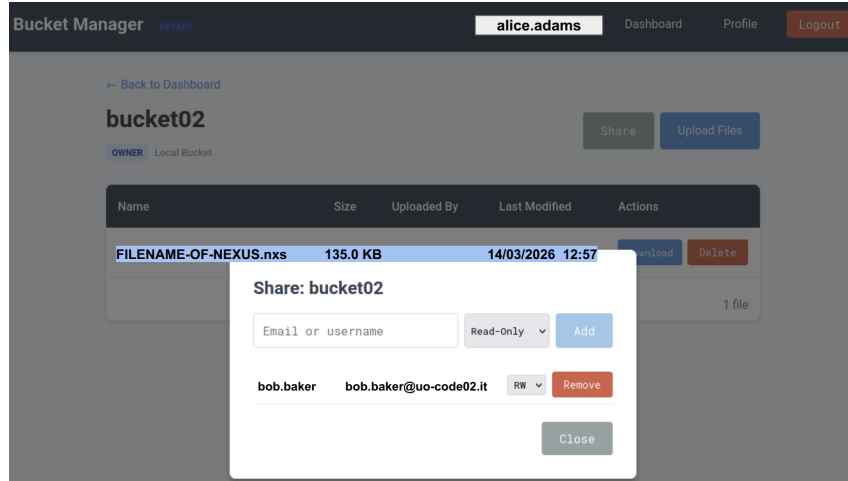
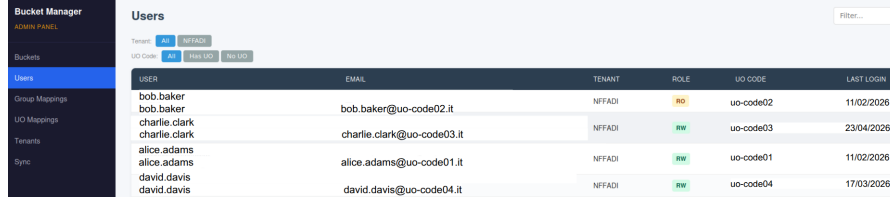


Figure A.4: Bucket sharing dialog. The owner can add collaborators by email or username and assign read-only or read-write permission per user. Existing collaborators are listed with their current permission level and can be removed. The change propagates from the application database to Ceph through per-user S3 credential updates in the Django backend.

NAME	TENANT	TYPE	STORAGE 1	OBJECTS 1	SHARES 1	CREATED 1
420	NFFADI	PROPOSAL	0 B	0	6	11/03/2026
6 users with access						
frank.fisher	frank.fisher	frank.fisher@uo-code06.it				RW
eve.evans	eve.evans	eve.evans@uo-code06.it				RW
david.davis	david.davis	david.davis@uo-code04.it				RW
charlie.clark	charlie.clark	charlie.clark@uo-code03.it				RW
alice.adams	alice.adams	alice.adams@uo-code01.it				RW
bob.baker	bob.baker	bob.baker@uo-code02.it				RO
bucket02	NFFADIALice-adams-cnrr-fn-tn-bucket02	LOCAL	0 B	0	2	12/03/2026
2 users with access						
bob.baker	bob.baker	bob.baker@uo-code02.it				RO
alice.adams	alice.adams	alice.adams@uo-code01.it				RW

Figure A.5: Administrator bucket panel. The admin view lists all buckets across tenants with type badges (PROPOSAL or LOCAL), full bucket names that embed the **UO** code for **NFFA-DI** local buckets, storage usage, and per-bucket user access lists. The sidebar gives access to group mappings, **UO** mappings, tenant configuration, and the synchronization trigger.



USER	EMAIL	TENANT	ROLE	UO CODE	LAST LOGIN
bob.baker	bob.baker@uo-code02.it	NFFADI	RO	uo-code02	11/02/2026
bob.baker	bob.baker@uo-code02.it	NFFADI	RO	uo-code02	11/02/2026
charlie.clark	charlie.clark@uo-code03.it	NFFADI	RW	uo-code03	23/04/2026
charlie.clark	charlie.clark@uo-code03.it	NFFADI	RW	uo-code03	23/04/2026
alice.adams	alice.adams@uo-code01.it	NFFADI	RW	uo-code01	11/02/2026
alice.adams	alice.adams@uo-code01.it	NFFADI	RW	uo-code01	11/02/2026
david.davis	david.davis@uo-code04.it	NFFADI	RW	uo-code04	17/03/2026
david.davis	david.davis@uo-code04.it	NFFADI	RW	uo-code04	17/03/2026

Figure A.6: Administrator user panel. The admin view lists all registered users with their tenant memberships and group assignments. Administrators can inspect per-user access status, review group-to-tenant mappings, and manage user access across all tenants from a single interface.

A.3 SEM Classifier API: Technical Details

The SEM Classifier API has fewer deployment-state mechanics than the Bucket Manager because it does not bridge two governance paths or synchronize external state. The technical detail that belongs in the appendix concentrates in three places: the inference class hierarchy that separates the model-specific core from the reusable pipeline, the usage schema that records who called the service for what, and the exact request and response shapes a client must follow.

A.3.1 Inference Class Hierarchy

Section 4.3.3 described the three-level class structure in narrative form; the contract it establishes is summarized here for reference.

`BaseAsyncModelService` owns everything that does not depend on the model domain: the Redis connection, the background worker thread, and the job-lifecycle transitions between `PENDING`, `PROCESSING`, `COMPLETED`, and `FAILED`. A concrete subclass must implement four operations: `load_model()` (called once at startup), `encode_input(payload)` (turn caller input into a queue-safe representation), `decode_input(serialized)` (reconstruct the input on the worker side), and `predict(decoded)` (run inference and return a serializable result).

`ImageAsyncModelService` specializes that contract for image inputs. It resolves `URL`-based and binary-upload submissions into a uniform in-memory image before passing control to the base-class machinery. Any image-based analysis service can inherit this layer without reimplementing image-handling logic.

`SEMInferenceRedisService` is the concrete instantiation for `SEM` classification. It loads the `ViT` model and its preprocessing pipeline at startup,

runs inference inside `predict()`, and returns the predicted label, confidence score, and full probability distribution. Replacing it with another image classifier requires only a new concrete class that satisfies the same four-operation contract; the queue, worker loop, gateway, and usage tracking remain untouched.

A.3.2 Usage-Tracking Schema

The `api_usage` table (Figure 4.4 in Chapter 4) is deliberately denormalized: it stores `endpoint_type` as one of three logical values (`inference`, `job_status`, `job_results`) rather than the raw `URL` path. This is the right trade-off for the workload because operator queries repeatedly ask for counts by user, endpoint type, and time window; a categorical column lets the database planner use the `username` and `timestamp` indexes directly without parsing path strings.

The two secondary indexes are tuned for the two dominant access patterns: `timestamp` for time-range audits and per-day or per-hour aggregates, and `username` for per-user activity reports. The Go plugin writes each row in a goroutine after the HTTP response has already been sent, so database latency does not affect the client response; the operator reporting script (Section 4.6) reads the same table at any later time.

A.3.3 Request and Response Shapes

A client submits an inference request by POSTing to `/api/v1/inference` with either a `JSON` body containing an `image_url`, or a multipart form body containing an `image` file:

```
POST /api/v1/inference
Authorization: Bearer <jwt>
Content-Type: application/json
```

```
{ "image_url": "https://example.org/sem-particles.jpg" }
```

The gateway returns a job identifier immediately:

```
{ "job_id": "5f9c3a2e-...-b1a4" }
```

The client then polls `/api/v1/jobs/status` until the response reports a terminal state:

```
{
  "job_id": "5f9c3a2e-...-b1a4",
  "status": "COMPLETED",
  "timestamps": {
    "submitted": "2026-04-12T10:14:32Z",
    "started":   "2026-04-12T10:14:33Z",
    "completed": "2026-04-12T10:14:35Z"
  }
}
```

A POST to `/api/v1/jobs/results` with the same `job_id` retrieves the classification output:

```
{
  "label": "Particles",
  "confidence": 0.9798,
  "all_scores": {
    "Biological": 0.001, "Fibres": 0.002,
    "Films": 0.003, "MEMS": 0.005,
    "Nanowires": 0.004, "Particles": 0.9798,
    "Patterned": 0.001, "Porous": 0.002,
    "Powders": 0.001, "Tips": 0.001
  }
}
```

A new service built on the same template exposes the same request life-cycle: only the `predict()` method and the result schema change.

A.4 Infrastructure Component Notes

A.4.1 Kubernetes Orchestration

The storage and analysis services that appear as coherent architecture blocks are, at runtime, collections of containers with separate lifecycles. Containers package code and dependencies so that one component runs consistently, but packaging does not answer operational questions: placement, instance count, discovery, restart, and update policy. Those are orchestration problems.

An orchestration system coordinates those components so that the collection behaves as one service. Kubernetes provides that coordination through a desired-state control model. The deployment specification is the set of Kubernetes manifests, namely configuration files that declare which container

images should run, the desired instance count, which stable service names should exist, which requests for persistent storage should be attached, and which readiness checks must pass before traffic is sent. Kubernetes compares that declared state with the actual cluster state and acts to close the gap.

That control model matters because the thesis projects are service compositions. The Bucket Manager combines browser-facing frontend logic, backend authorization, identity integration, application state, and storage-control operations. The analysis-service pipeline combines a gateway, inference service, queue, and usage database. Kubernetes gives these components a shared operational boundary: each part can have its own runtime needs while still belonging to one deployed service.

The boundary depends on a few high-level Kubernetes mechanisms. Frontend, backend, gateway, and inference containers do not own state, so Kubernetes can replace them as service logic. Redis and PostgreSQL carry state, so they require persistent storage and more conservative lifecycle handling. Service objects give components stable names instead of transient container identities; readiness checks keep traffic away from containers that have not finished starting; rolling replacement allows a gateway, frontend, or inference component to change without turning the whole service off. These mechanisms let the Bucket Manager behave as a storage-access layer and the analysis pipeline behave as an **OFED** service rather than as a local prototype.

A.5 Development Environment

The projects were developed and tested in a virtual data-center environment rather than directly against the **ORFEO** production cluster. The environment was provisioned by Simulated Testing Environment for Nibble Cluster Infrastructure Lab (**STENCIL**), a virtual data-center framework documented through its public project page and GitLab source namespace [15, 16]. The framework deploys a self-contained virtual data center inside a single virtual machine. A dedicated VM running inside the **ORFEO** infrastructure hosted this setup as the development substrate.

The VM provisions four coordinated layers that together replicate the production environment. The first layer is virtualization: the VM itself runs on the **ORFEO** hardware and acts as an isolated boundary for everything that follows. The second layer is a local Ceph cluster, providing S3-compatible object storage that mirrors the production Ceph **RGW**—this allows the bucket-manager application to interact with a real S3-compatible backend without touching shared production buckets. The third layer is FreeIPA—an open-source identity management system bundling DNS, Ker-

beros authentication, and LDAP—which provides internal DNS resolution and Kerberos-based identity services for the cluster; this layer is needed because service-to-service calls within Kubernetes rely on stable DNS names, and authentication flows require a functioning identity issuer. The fourth layer is a lightweight Kubernetes distribution (**K3s**) cluster that runs on top of the identity and storage layers and hosts the deployed application services.

The practical consequence is that the thesis deployments were exercised against real orchestration, persistent storage, readiness, rolling-update, and **JWT**-authentication paths before any production path was considered. The **STENCIL** environment kept development iterations fast and failures contained.

Figure A.7 shows the resulting stack: **ORFEO** hardware hosts the **STENCIL** VM, the VM provides storage and identity services, and the **K3s** cluster runs the two thesis services above those foundations.

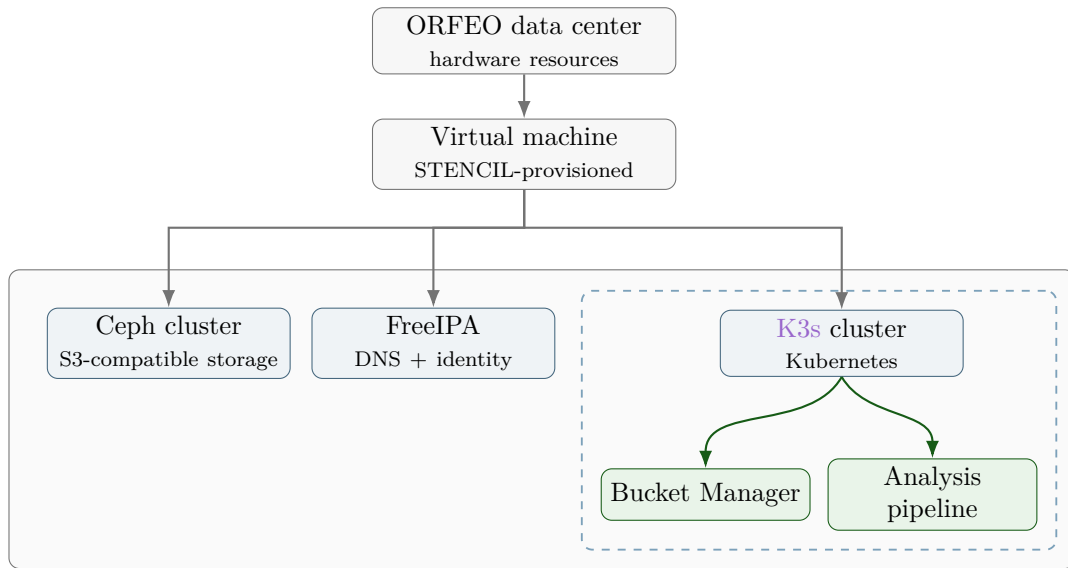


Figure A.7: Development environment structure. A **STENCIL**-provisioned virtual machine inside **ORFEO** hosts a local Ceph cluster, FreeIPA, and a **K3s** cluster. The thesis projects were deployed, tested, and iterated inside that **K3s** cluster before any production path was considered.

Bibliography

- [1] NFFA-DI, “Data infrastructure.” Nffa-di official website, 2025.
<https://nffa-di.it/en/data-infrastructure/>. Accessed April 20, 2026.
- [2] F. Bazzocchi, “Data management in NFFA-DI: OFED (Overarching FAIR Ecosystem for Data).” Presentation, zenodo, 2024.
<https://zenodo.org/records/13773924>.
- [3] NFFA-DI, “Research data policy.” Nffa-di official website, 2025.
<https://nffa-di.it/en/research-data-policy>. Last updated March 25, 2025; accessed April 20, 2026.
- [4] M. Marella, “Implementing a FAIR-by-design approach for process management in nanoscience foundries,” Master’s thesis, SISSA - Scuola Internazionale Superiore di Studi Avanzati, Master in Data Management and Curation, 2025.
<https://zenodo.org/records/15678583>.
- [5] S. Vigneri, “Design refinement and commissioning of a FAIR-by-design integrated data management system for an STM laboratory,” Master’s thesis, SISSA - Scuola Internazionale Superiore di Studi Avanzati, Master in Data Management and Curation, 2025.
<https://zenodo.org/records/15664063>.
- [6] E. Saadat, “Implementation of a pipeline for collecting, ingesting and transforming data into standard formats for the LAME FIB-SEM,” Master’s thesis, SISSA - Scuola Internazionale Superiore di Studi Avanzati, Master in Data Management and Curation, 2025.
<https://zenodo.org/records/15708394>.
- [7] N. Perin, “A FAIR data ecosystem in transmission electron microscopy,” Master’s thesis, SISSA - Scuola Internazionale Superiore di Studi Avanzati, Master in Data Management and Curation, 2025.
<https://doi.org/10.5281/zenodo.15707274>.

- [8] S. Higgins, “The DCC curation lifecycle model,” *International Journal of Digital Curation* **3** no. 1, (2008) 134–140.
<https://doi.org/10.2218/ijdc.v3i1.48>.
- [9] University of Reading, “The research data lifecycle.” University of reading research data management guidance, 2026.
<https://cms9-prod-ce.rdg.ac.uk/research-services/research-data-management/about-research-data-management/the-research-data-lifecycle>. Accessed April 21, 2026.
- [10] AREA Science Park, “ORFEO: High-performance computing and data infrastructure.” Orfeo official documentation, 2026.
<https://orfeo-doc.areasciencepark.it/>. ORFEO is the high-performance computing and data infrastructure operated by AREA Science Park in Trieste, Italy. Accessed May 2026.
- [11] Amazon Web Services, “What is amazon s3?” Amazon s3 official documentation, 2026. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>. Accessed April 20, 2026.
- [12] Ceph, “Ceph object gateway.” Ceph official documentation, 2026.
<https://docs.ceph.com/en/squid/radosgw/>. Accessed April 20, 2026.
- [13] authentik, “Welcome to authentik.” Authentik official documentation, 2026. <https://docs.goauthentik.io/>. Accessed April 20, 2026.
- [14] Kubernetes, “Overview.” Kubernetes official documentation, 2026.
<https://kubernetes.io/docs/concepts/overview/>. Accessed April 20, 2026.
- [15] I. Pasianotto, “STENCIL: Simulated testing environment for nibble cluster infrastructure lab.” Project webpage, 2026.
<https://isacpasianotto.com/projects/stencil/>. Public project page documenting STENCIL as a virtual data-centre and virtual cluster framework. Accessed May 17, 2026.
- [16] Project STENCIL, “Project STENCIL gitlab namespace.” Gitlab source-code namespace, 2026. <https://gitlab.com/projectstencil>. Public source-code namespace for the STENCIL project. Accessed May 17, 2026.
- [17] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos,

- P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, and B. Mons, “The FAIR guiding principles for scientific data management and stewardship,” *Scientific Data* **3** (2016) 160018.
<https://doi.org/10.1038/sdata.2016.18>.
- [18] MinIO, “MinIO AISTor: Enterprise object storage platform.” Minio official documentation, 2025. <https://docs.min.io/enterprise/aistor-object-store/administration/iam/access/oidc-access/>. OpenID Connect (OIDC) and single sign-on (SSO) identity federation, previously available in the open-source AGPL community edition of MinIO Server, are documented exclusively under the MinIO AISTor enterprise product as of April 2025. Accessed May 10, 2026.
- [19] Amazon Web Services, “Boto3: AWS SDK for Python.” Aws official documentation, 2026. <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>. Boto3 is the official AWS SDK for Python, providing a Python API for Amazon S3-compatible object storage operations including bucket management, object upload and download, and presigned URL generation. Accessed May 10, 2026.
- [20] NFFA-DI, “NFFA-DI consortium partners.” Nffa-di official website, 2026. <https://nffa-di.it/en/about-us/consortium/>. NFFA-DI comprises eleven Italian operational units spanning CNR institutes, AREA Science Park, Politecnico di Milano, and Università degli Studi di Milano. Accessed May 10, 2026.
- [21] NFFA-DI, “Uo acronyms for data infrastructure naming convention.” Nffa-di official website, 2025. <https://nffa-di.it/en/uo-acronyms-for-data-infrastructure-naming-convention>. Last updated March 25, 2025; accessed April 20, 2026.
- [22] CERIC-ERIC, “Central European research infrastructure consortium.” Ceric-eric official website, 2026. <https://www.ceric-eric.eu/about-us/>. CERIC-ERIC (Central

- European Research Infrastructure Consortium) is a European research infrastructure established in 2014, headquartered at AREA Science Park in Trieste, Italy. The PRP@CERIC (Pathogen Readiness Platform for CERIC-ERIC) programme extends access to CERIC-affiliated facilities and storage infrastructure. Accessed May 10, 2026.
- [23] Django Software Foundation, “The web framework for perfectionists with deadlines.” Django official documentation, 2026. <https://www.djangoproject.com/>. Accessed May 10, 2026.
- [24] NeXus International Advisory Committee, “NeXus: A common data format for neutron, x-ray, and muon science.” Nexus official documentation, 2026. <https://www.nexusformat.org/>. Open file-format standard based on HDF5, widely used in photon-science, neutron-scattering, and materials science experiments. Accessed May 2026.
- [25] Meta Open Source, “React: The library for web and native user interfaces.” React official documentation, 2026. <https://react.dev/>. Accessed May 10, 2026.
- [26] silx-kit, “H5Web: Web-based hdf5 data viewer.” Github repository and documentation, 2026. <https://github.com/silx-kit/h5web>. Open-source React-based viewer for HDF5 and NeXus data files. Accessed May 2026.
- [27] PostgreSQL Global Development Group, “About postgresql.” Postgresql official website, 2026. <https://www.postgresql.org/about/>. Accessed April 20, 2026.
- [28] t0m-R, “vit-sem-classification: Vision Transformer for SEM image classification.” Hugging face model repository, 2025. <https://huggingface.co/t0m-R/vit-sem-classification>. ViT fine-tuned on NFFA-DI scanning electron microscopy images for materials science image classification. Apache 2.0 licence. Accessed May 2026.
- [29] R. Aversa, M. H. Modarres, S. Cozzini, R. Ciancio, and A. Chiusole, “The first annotated set of scanning electron microscopy images for nanoscience,” *Scientific Data* **5** (2018) 180172. <https://doi.org/10.1038/sdata.2018.172>.

- [30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*. 2021.
<https://openreview.net/forum?id=YicbFdNTTy>.
- [31] NOMAD Laboratory, “NOMAD: Open materials-science data platform.” Nomad laboratory official website, 2026.
<https://nomad-lab.eu/>. NOMAD is an open data platform for computational and experimental materials science, aggregating data from heterogeneous scientific sources and supporting reuse, curation, and AI-based analysis. Accessed May 2026.
- [32] NFFA-Europe, “TriDAS: Trieste advanced data services.” Nffa-europe official website, 2026.
<https://nffa.eu/apply/virtual-access-services/tridas/>. TriDAS is described as a collection of data services based on data analytics and machine learning, including a SEM Classifier service. Accessed May 2026.
- [33] KrakenD, “Krakend documentation.” Krakend official documentation, 2026. <https://www.krakend.io/docs/>. Accessed April 20, 2026.
- [34] BentoML, “Bentoml documentation.” Bentoml official documentation, 2026. <https://docs.bentoml.com/>. Accessed April 20, 2026.
- [35] Redis, “Open source.” Redis official documentation, 2026.
<https://redis.io/docs/latest/get-started/>. Accessed April 20, 2026.
- [36] BentoML, “Services and runners — BentoML documentation.” Bentoml official documentation (v1.4), 2026.
<https://docs.bentoml.com/en/latest/guides/services.html>. Service methods decorated with `@bentoml.api` are exposed as HTTP POST endpoints by default. The project uses BentoML 1.4.36. Accessed May 2026.
- [37] KrakenD, “Endpoint configuration.” Krakend official documentation, 2026. <https://www.krakend.io/docs/endpoints/>. KrakenD routes are defined by URL path prefix; URI versioning (`/api/v1/`, `/api/v2/`) is the conventional mechanism for coexisting API versions. Accessed May 2026.

- [38] t0m-R, “vit-stm-artifact-fft: Vision Transformer for STM multi-tip artifact detection.” Hugging face model repository, 2025.
<https://huggingface.co/t0m-R/vit-stm-artifact-fft>.
Fine-tuned Vision Transformer for classifying STM images as artifact-free or multi-tip artifacts. Apache 2.0 licence. Accessed May 2026.
- [39] t0m-R, “vit-sem-scale-classifier: Vision Transformer for SEM scale classification.” Hugging face model repository, 2025.
<https://huggingface.co/t0m-R/vit-sem-scale-classifier>.
Image-classification model repository for SEM scale classification. Accessed May 2026.
- [40] The Go Authors, “The Go programming language documentation.” Go official documentation, 2026. <https://go.dev/doc/>. Go is a statically typed, compiled programming language developed at Google. Accessed May 2026.
- [41] KrakenD, “Extending KrakenD: Custom plugins.” Krakend official documentation, 2026. <https://www.krakend.io/docs/extending/>.
KrakenD’s plugin system allows Go shared libraries to be loaded at startup and wired into the HTTP handler chain without modifying the gateway’s own source code. Accessed May 2026.
- [42] The Go Authors, “Package plugin.” Go standard library reference, 2026. <https://pkg.go.dev/plugin>. The `plugin` package implements loading of Go plugins (shared objects compiled from Go source) at runtime. Accessed May 2026.
- [43] Hugging Face, “Hugging Face Hub: A platform for sharing machine learning models and datasets.” Hugging face official website, 2026. <https://huggingface.co/>. Public repository for machine-learning models, datasets, and evaluation tools. Accessed May 2026.